

Dédicaces

Je dédie ce travail à mes parents, qui ont toujours été là pour moi. Leur amour, leur soutien et leurs efforts m'ont donné la force d'avancer et de croire en mes capacités. Sans eux, ce parcours aurait été beaucoup plus difficile.

À ma famille, pour leur présence rassurante et leurs encouragements. Leur soutien m'a aidé à rester motivé et à surmonter les moments compliqués de ce stage.

À mes amis, pour leurs conseils, leur bonne humeur et les moments qui m'ont permis de me détendre. Leur accompagnement m'a réellement aidé à garder l'équilibre durant cette période.

Enfin, à toutes les personnes qui m'ont aidé de près ou de loin tout au long de ce projet. Merci pour votre disponibilité, votre soutien et votre contribution à la réalisation de ce rapport.

Remerciements

Avant tout, je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réussite de ce stage et à la réalisation de ce rapport.

J'adresse mes sincères remerciements à **l'entreprise Vala Orange**, pour m'avoir accueilli au sein de son équipe et offert l'opportunité de participer à un projet aussi formateur, consacré à la conception d'un modèle de détection des fraudes bancaires à l'aide du machine learning.

Je tiens à remercier tout particulièrement **M. Neumann Charhbili**, mon encadrant au sein de l'entreprise, pour sa confiance, son accompagnement constant, sa disponibilité et la qualité de ses conseils qui m'ont permis de progresser tout au long de cette expérience.

Mes remerciements s'adressent également aux **membres du jury**, qui prendront le temps d'évaluer ce rapport et d'apprécier les efforts fournis durant ce stage. Leur attention et leurs observations seront pour moi une source précieuse d'apprentissage.

Enfin, je souhaite remercier l'ensemble du personnel de l'entreprise pour leur accueil chaleureux, leur collaboration et l'ambiance professionnelle stimulante qu'ils ont su instaurer.

À toutes ces personnes, je témoigne ma reconnaissance la plus sincère et ma profonde considération.

Résumé

Au cours de mon stage au sein de **Vala Orange**, j'ai eu l'opportunité de participer à un projet majeur : la détection des fraudes bancaires à l'aide de techniques de **Machine Learning**. Ce projet représente mon **premier véritable pas dans le monde de la data science**, me permettant de mettre en pratique mes connaissances sur des problématiques concrètes.

Mon travail a consisté à collecter et analyser des jeux de données transactionnelles, à concevoir des modèles prédictifs et à évaluer leur capacité à détecter des comportements suspects. Pour cela, j'ai utilisé différentes techniques et outils de la **data science** pour traiter, analyser et visualiser les données.

Les résultats obtenus montrent que le modèle développé est capable de **repérer efficacement les transactions frauduleuses**, offrant ainsi un outil potentiel pour renforcer la sécurité des opérations bancaires. Cette expérience m'a permis de consolider mes compétences en **Machine Learning** et en analyse de données, tout en participant à un projet concret et à fort impact.

Table des matières

Dédicaces	2
Remerciements	3
Liste des acronymes	3
Résumé	4
Table des matières	5
Table des figures	8
Liste des tableaux	10
Introduction Générale	11
1 Présentation de l'entreprise	12
1.1 Historique et activités principales	12
1.2 Organisation et structure	13
1.3 Services et technologies utilisées	14
1.4 Environnement de travail	15
1.4.1 Cadre physique	15
1.4.2 Organisation humaine	15
1.4.3 Outils et logiciels utilisés	15
1.4.4 Conditions de travail et communication	15
2 Présentation du projet de stage	17
2.1 Contexte et problématique	17
2.2 Objectifs du projet	18
2.3 Enjeux et motivations	19
2.4 Méthodologie adoptée	20
3 Conception du système	22
3.1 Analyse fonctionnelle	22

3.2	Architecture générale du projet	25
3.3	Modélisation des données	27
3.4	Choix technologiques	30
4	Réalisation et mise en œuvre	34
4.1	Développement web (interface et API)	34
4.1.1	Développement de l'interface utilisateur (Front-end)	35
4.1.2	Développement de l'API et du Back-end	40
4.1.3	Communication Front-end / Back-end	44
4.2	Exploration et Présentation des Données	45
4.2.1	Présentation du jeu de données	45
4.2.2	Analyse statistique globale	46
4.2.3	Analyse du déséquilibre des classes	47
4.2.4	Analyse par type de transaction	48
4.2.5	Analyse des corrélations entre variables	49
4.3	Pipeline de traitement des données	51
4.3.1	Structure générale et fonctionnement	51
4.3.2	Variables utilisées pour la prédiction	53
4.3.3	Implémentation du pipeline	53
4.3.4	Gestion des erreurs et cas limites	54
4.4	Création du modèle de Machine Learning	55
4.4.1	Méthodologie de détection et approche supervisée	55
4.4.2	Gestion du déséquilibre des classes	56
4.5	Entraînement et évaluation des modèles	57
4.5.1	Préparation des données	57
4.5.2	Métriques d'évaluation : pourquoi l'Accuracy ne suffit pas	58
4.5.3	Stratégie de validation croisée	59
4.5.4	Entraînement et configurations testées	59
4.5.5	Résultats de la validation croisée	59
4.5.6	Comparaison globale des configurations	62
4.5.7	Impact des techniques d'équilibrage	63
4.5.8	Analyse SMOTE vs Undersampling	64
4.5.9	Matrices de confusion	64
4.5.10	Courbes ROC	66
4.5.11	Analyse des erreurs du modèle XGBoost	66
4.6	Déploiement et intégration du modèle	67
4.6.1	Sérialisation et intégration du modèle ML	67
4.6.2	Déploiement conteneurisé avec Docker	68

4.6.3	Orchestration avec Docker Compose	69
5	Discussion et perspectives	71
5.1	Introduction	71
5.2	Difficultés rencontrées	71
5.3	Améliorations possibles	71
5.4	Perspectives d'évolution	72
5.5	Conclusion	72
	Conclusion Générale	73
	Références Bibliographiques	74

Table des figures

1.1	Organigramme de l'entreprise Vala Orange	14
3.1	Diagramme de cas d'utilisation du système de détection des fraudes bancaires	24
3.2	Diagramme d'architecture générale du système de détection des fraudes bancaires	27
3.3	Diagramme de classes du système de détection des fraudes bancaires	28
3.4	Diagramme de séquence du processus de prédiction d'une transaction unique	29
3.5	Diagramme de séquence du processus d'analyse d'un fichier CSV	30
4.1	Page d'accueil présentant le système avant authentification, offrant un accès rapide aux principales fonctionnalités	36
4.2	Exemple de composant React : Dashboard.jsx illustrant la modularité et la lisibilité du code front-end	37
4.3	Interface de connexion sécurisée avec JWT, offrant une expérience utilisateur claire et intuitive	37
4.4	Tableau de bord principal affichant les statistiques de détection en temps réel, conçu pour la lisibilité et l'interactivité	38
4.5	Page de prédiction des fraudes via formulaire ou fichier CSV, offrant plusieurs options d'entrée pour l'utilisateur	38
4.6	Formulaire manuel pour la prédiction des fraudes, facilitant l'entrée individuelle de transactions	39
4.7	Résultats des prédictions à partir d'un fichier CSV, présentés de manière claire et structurée	39
4.8	Résultats des prédictions via formulaire manuel, avec mise en évidence des transactions suspectes	40
4.9	Page d'administration pour la gestion des modèles et utilisateurs, offrant un contrôle complet du système	40
4.10	Documentation automatique des endpoints via Swagger, facilitant la compréhension et les tests	42
4.11	Test de l'API de prédiction à l'aide de Postman pour vérifier les réponses et la conformité des endpoints	42

4.12	Structure générale du projet Back-end FastAPI, illustrant la modularité et la lisibilité du code	43
4.13	Exemple de code : routes du module <code>transactions</code> , démontrant la séparation des responsabilités	43
4.14	Exemple de code : services du module <code>transactions</code> , illustrant la logique métier centralisée	44
4.15	Répartition des transactions selon la classe <code>isFraud</code>	47
4.16	Distribution des montants de transaction.	48
4.17	Taux de fraude par type de transaction, montrant la concentration sur TRANSFER et CASH_OUT.	49
4.18	Heatmap des corrélations entre les principales variables numériques du dataset.	50
4.19	Extrait de code : chargement et validation initiale des données	53
4.20	Extrait de code : création des variables dérivées pour la détection de fraudes	54
4.21	Extrait de code : transformation des données et prédiction par le modèle .	54
4.22	Distribution des scores F1 de la Régression Logistique sur les 5 folds pour chaque configuration.	60
4.23	Distribution des scores F1 de Random Forest sur les 5 folds pour chaque configuration.	61
4.24	Distribution des scores F1 de XGBoost sur les 5 folds pour chaque configuration.	62
4.25	Comparaison visuelle des performances (F1-Score) des trois modèles avec leurs meilleures configurations.	63
4.26	Impact des techniques d'équilibrage sur le Recall de chaque modèle.	64
4.27	Matrices de confusion des trois modèles avec SMOTE sur l'ensemble de test.	65
4.28	Courbes ROC des trois modèles avec SMOTE et leurs scores AUC respectifs.	66
4.29	Image Docker du backend	69
4.30	Fichier <code>docker-compose.yaml</code>	70

Liste des tableaux

3.1	Technologies utilisées pour le système web	31
3.2	Bibliothèques de Machine Learning	31
4.1	Aperçu des cinq premières transactions du dataset	46
4.2	Statistiques descriptives des variables numériques du dataset	46
4.3	Variables utilisées pour la prédiction des fraudes bancaires	53
4.4	Performances de la Régression Logistique avec validation croisée (5 folds) .	59
4.5	Performances de Random Forest avec validation croisée (5 folds)	60
4.6	Performances de XGBoost avec validation croisée (5 folds)	61
4.7	Comparaison des meilleures configurations pour chaque modèle	62
4.8	Amélioration du Recall grâce aux techniques d'équilibrage	63

Introduction Générale

Dans le cadre de mon stage d'initiation, j'ai eu l'opportunité de m'impliquer dans un projet concret au sein de l'entreprise **Vala Orange**, agence digitale spécialisée dans le développement web, le SEO et les solutions digitales sur mesure. Ce stage m'a offert une immersion précieuse dans le fonctionnement d'une entreprise digitale et m'a permis de découvrir un domaine qui m'était jusqu'alors peu familier : la data science appliquée à des problématiques réelles.

Lors de ce stage, mon encadrant m'a donné toute la liberté de choisir un projet personnel, ce qui m'a permis d'explorer le monde de la data science selon mes propres intérêts. J'ai ainsi décidé de me concentrer sur la **détection des fraudes bancaires par Machine Learning**, un projet stimulant qui m'a permis de combiner programmation, analyse de données et apprentissage automatique.

Mon travail a consisté à **collecter et analyser des jeux de données transactionnelles provenant de la plateforme Kaggle**, à les prétraiter et à explorer les données, puis à développer différents modèles prédictifs à l'aide de Python et des bibliothèques de science de données. Cette expérience m'a permis de consolider mes compétences techniques tout en apprenant à aborder un projet complexe de manière structurée et rigoureuse.

Ce rapport constitue la synthèse de ce projet et a pour objectif de présenter l'ensemble des travaux réalisés, les résultats obtenus ainsi que les compétences développées au cours du stage. Il est rédigé de manière à offrir une vision progressive et cohérente du projet et des apprentissages associés.

Ainsi, le rapport est organisé comme suit : le premier chapitre présente l'entreprise et son environnement, le second est consacré à la présentation du projet de stage, le troisième décrit la conception du système, tandis que le quatrième détaille sa réalisation et sa mise en œuvre. Enfin, le dernier chapitre propose une discussion des résultats obtenus ainsi que des perspectives d'amélioration.

1. Présentation de l'entreprise

Introduction du chapitre

Ce chapitre a pour objectif de présenter l'entreprise **Vala Orange**, son historique, ses activités principales, son organisation interne, les services et technologies utilisés, ainsi que l'environnement de travail dans lequel j'ai effectué mon stage. Cette présentation permet de mieux comprendre le fonctionnement global de l'entreprise et le contexte dans lequel mon projet en data science a été réalisé.

1.1 Historique et activités principales

La société **Vala Orange** a été créée le 30 septembre 2020 sous la forme juridique d'une S.A.R.L., sous la direction de M. Kayouh Abdelbast. L'entreprise compte aujourd'hui une équipe de 11 collaborateurs. Située à l'Immeuble Safwa, Boulevard Hassan 1er, 2^e étage N°6, à Dakhla – Agadir, elle s'est rapidement imposée comme un acteur dynamique dans le domaine de la production digitale, offrant des solutions numériques complètes et adaptées aux besoins variés de sa clientèle.

Depuis sa création, **Vala Orange** accompagne ses clients dans leur transformation digitale en proposant des services personnalisés, flexibles et innovants. L'entreprise collabore avec des structures diverses, allant de petites entreprises locales à des organisations nécessitant des solutions plus complexes, et place la satisfaction client au cœur de sa démarche. Cette approche, centrée sur la qualité et la créativité, lui a permis de se forger une réputation solide sur le marché local.

Les activités de l'entreprise couvrent un large éventail de prestations digitales : développement et hébergement de sites web, conception d'applications web et mobiles, optimisation SEO et marketing digital, ainsi que design d'interfaces ergonomiques et attractives. En s'appuyant sur une équipe pluridisciplinaire, **Vala Orange** gère les projets de A à Z, de la conception initiale à la livraison finale, tout en assurant un suivi rigoureux et une

maintenance adaptée.

1.2 Organisation et structure

Afin d'assurer une gestion efficace et une répartition claire des responsabilités, l'entreprise est structurée en plusieurs départements fonctionnels sous une hiérarchie simple mais organisée. Au sommet se trouve le Directeur Général, M. Abdelbast Kayouh, qui supervise l'ensemble des activités et définit les orientations stratégiques. Il est assisté par une assistante de direction, Asma El Boudi, assurant la coordination administrative et le suivi des projets.

Les principaux départements fonctionnels sont :

- **Développement Web** : dirigé par Bachir El Omari, ce département conçoit et développe les sites web et applications, en veillant à la qualité, à la sécurité et à la performance des solutions livrées.
- **Direction de projets Web** : sous la responsabilité de Khalid Jbour, ce service coordonne les projets numériques, planifie les étapes de réalisation et assure la liaison avec les clients. Il inclut également le Responsable Service Après-Vente (Wafa Fouks), le Développeur SEO (Neumann Charhbili) et la Rédactrice Web SEO (Laila Isegesioui).
- **Service Commercial** : dirigé par Fatiha Disky, il est en charge du suivi commercial, de la relation client et de la prospection de nouveaux marchés.
- **Service Client** : sous la direction de Hanane Bayen, ce département garantit la satisfaction des clients après la livraison des projets, avec un suivi assuré par Oumaima Ait Ihya.
- **Administration et Finances** : dirigé par Soukaina Ouahmid, responsable de la gestion administrative, financière et comptable.

Cette structure hiérarchique, tout en restant flexible, favorise la collaboration interservices et assure une coordination fluide des projets. Durant mon stage, elle m'a permis de comprendre le rôle de chaque département et de travailler en interaction avec différents services.

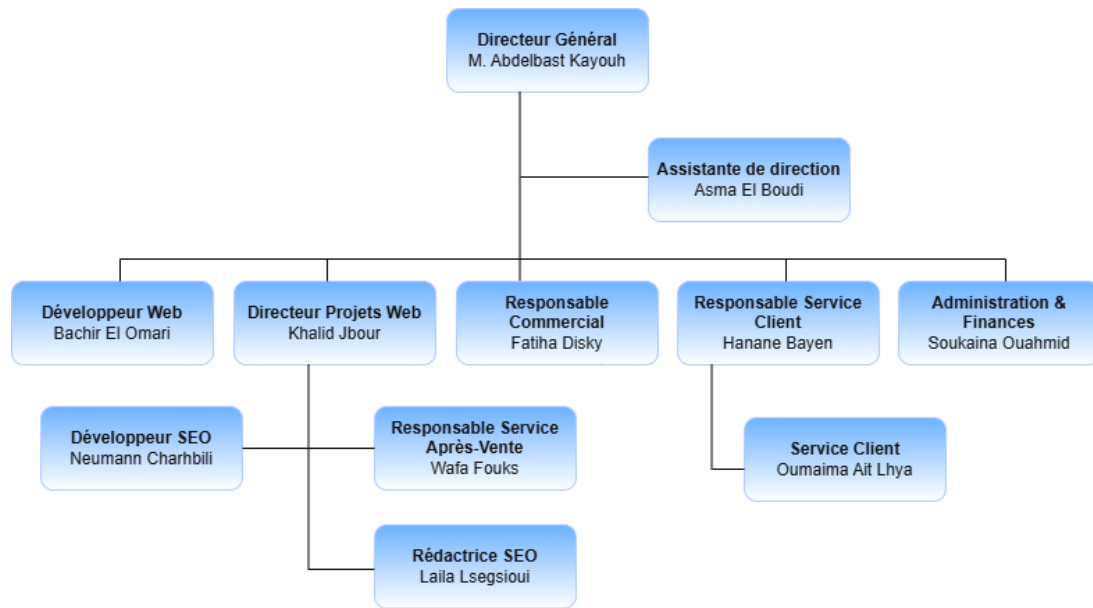


FIGURE 1.1 – Organigramme de l’entreprise Vala Orange

1.3 Services et technologies utilisées

Les services proposés par **Vala Orange** s’appuient sur une infrastructure moderne et sur des compétences variées, permettant de couvrir l’ensemble du cycle de vie d’un projet numérique.

1. Développement Web et Hébergement

Le département de développement conçoit et réalise des sites web et applications sur mesure, allant des sites vitrines aux boutiques en ligne et applications métiers. Chaque projet est pensé pour être rapide, sécurisé et ergonomique. Les solutions sont hébergées sur des serveurs performants, avec Cloud et Load Balancing, garantissant haute disponibilité et scalabilité. Les clusters sont répartis sur plusieurs datacenters pour assurer sécurité et performance.

2. Messagerie professionnelle et gestion des domaines

Vala Orange fournit des solutions de messagerie sécurisée, anti-spam et anti-virus, synchronisées sur plusieurs appareils, avec gestion complète des noms de domaine. Les communications sont protégées par des certificats TLS et les protocoles standards IMAP/POP/SMTP.

3. Optimisation SEO, marketing digital et e-commerce

Pour améliorer la visibilité en ligne, l’entreprise propose SEO, optimisation des sites et création de boutiques en ligne. Les technologies utilisées incluent WordPress, WooCommerce, bases de données MariaDB/Cluster, systèmes de cache et CDN pour accélérer le contenu. L’approche est mobile-first et sécurisée pour les paiements.

4. Sécurité et maintenance

Les projets sont protégés par des sauvegardes régulières, pare-feu, audits de sécurité et

monitoring permanent, garantissant fiabilité et continuité des services.

Ces outils et technologies ont constitué le socle sur lequel s'est appuyé mon travail de stage, en particulier dans le développement web et l'optimisation des performances des projets numériques.

1.4 Environnement de travail

1.4.1 Cadre physique

Le service où j'ai effectué mon stage occupe le deuxième étage de l'Immeuble Safwa, dans un bâtiment moderne. Il comprend des bureaux individuels favorisant la concentration, ainsi qu'un espace de travail commun pour les réunions et échanges. Chaque poste est équipé d'ordinateurs performants, même si j'ai principalement travaillé sur mon propre ordinateur.

Bien que l'entreprise ne dispose pas directement de salle de pause, elle collabore avec un établissement à proximité, offrant un lieu agréable pour les moments de détente et les échanges informels.

1.4.2 Organisation humaine

Durant mon stage, j'ai travaillé de manière autonome tout en bénéficiant de l'encadrement de Neumann Charhbili lorsque nécessaire. La hiérarchie claire et la disponibilité des responsables permettent un encadrement structuré tout en offrant autonomie et responsabilités aux stagiaires.

La culture de Vala Orange encourage le partage d'informations, la transmission des bonnes pratiques et les échanges professionnels, facilitant ainsi l'intégration et le développement des compétences.

1.4.3 Outils et logiciels utilisés

Pour le suivi et l'organisation de mon projet, j'ai utilisé **GitHub** pour centraliser le code et partager le lien avec mon encadrant, et **Visual Studio Code (VSCode)** comme IDE pour coder efficacement et gérer les fichiers de manière organisée.

1.4.4 Conditions de travail et communication

La présence quotidienne, soit le matin soit l'après-midi, permet de concilier travail en présentiel et à distance. Les horaires officiels sont du lundi au vendredi de 9h à 16h30, et le samedi de 9h à 13h.

Le suivi des projets se fait à travers des réunions hebdomadaires et via GitHub, où les mises à jour du projet sont visibles en temps réel pour l'encadrant.

Conclusion du chapitre

Ce chapitre a présenté de manière détaillée l'entreprise **Vala Orange**, son historique, ses services, ses technologies et son organisation interne. L'analyse de son environnement de travail et de sa structure hiérarchique a permis de mieux comprendre le cadre dans lequel mon projet de stage a été mené, et de mettre en évidence l'importance de la collaboration, de l'encadrement et de l'utilisation d'outils technologiques adaptés pour la réussite des projets numériques.

2. Présentation du projet de stage

Introduction du chapitre

Ce chapitre présente le cadre général du projet de stage, depuis le contexte professionnel jusqu'à la méthodologie suivie pour sa mise en œuvre. Il expose d'abord le contexte dans lequel le stage s'est déroulé et la problématique abordée, avant de définir les objectifs généraux et spécifiques du projet. Ensuite, les enjeux et motivations qui ont guidé le choix de ce sujet sont détaillés, mettant en lumière son importance technique, économique et personnelle. Enfin, la méthodologie adoptée est décrite, retraçant les différentes étapes du travail accompli, de la formation initiale à l'intégration du modèle dans une application web interactive.

2.1 Contexte et problématique

Contexte :

Dans le cadre de mon stage d'initiation chez **Vala Orange**, j'ai évolué dans un environnement professionnel stimulant, tout en bénéficiant de la liberté de choisir le sujet de mon projet. L'entreprise m'a offert un cadre favorable à l'apprentissage, où j'ai pu développer mes compétences de manière autonome, tout en recevant les conseils et le suivi régulier de mon encadrant, M. Neumann Charhbili, responsable technique. Cet accompagnement m'a permis de progresser efficacement et d'appliquer les bonnes pratiques professionnelles tout au long de la réalisation de mon projet.

Choix du projet :

J'ai choisi de travailler sur le développement d'un modèle de **Machine Learning pour la détection des fraudes bancaires**, un sujet qui combine mes intérêts pour le développement web et la data science. À l'ère du numérique, les transactions financières électroniques se multiplient rapidement, apportant commodité et efficacité dans les échanges économiques. Cependant, cette croissance s'accompagne d'une augmentation des fraudes bancaires, représentant un véritable défi pour les institutions financières. Selon des études récentes, les pertes liées à ces fraudes dépassent plusieurs milliards de dollars chaque

année, ce qui souligne l'importance de mettre en place des solutions fiables et innovantes.

Problématique :

L'objectif central de ce projet est de concevoir un système capable de détecter les comportements frauduleux de manière précise et rapide, tout en s'adaptant aux nouvelles techniques de fraude sans nécessiter d'intervention humaine constante. Un des principaux défis réside dans le traitement de données fortement déséquilibrées, les transactions frauduleuses ne représentant qu'une faible proportion du volume total. Cette situation rend la détection complexe, car il faut minimiser les faux positifs tout en maximisant la détection des fraudes. Dans le contexte marocain, où les systèmes de contrôle traditionnels reposent souvent sur des règles fixes et des vérifications manuelles, l'intégration de techniques d'intelligence artificielle et de Machine Learning apparaît comme une solution stratégique pour renforcer la sécurité et la fiabilité des transactions financières.

Cette problématique conduit naturellement à la définition d'objectifs précis pour le projet, qui orienteront la méthodologie et détermineront les résultats attendus.

2.2 Objectifs du projet

Objectif général :

L'objectif principal de ce projet est de concevoir et développer un modèle de Machine Learning capable de détecter efficacement les fraudes bancaires, en améliorant la précision et la rapidité de la détection tout en réduisant le besoin d'intervention humaine. Ce modèle sera intégré au sein d'une application web intuitive, permettant de visualiser les résultats et de faciliter la prise de décision.

Objectifs spécifiques :

- Collecter, nettoyer et explorer les données de transactions bancaires afin de constituer un jeu de données pertinent pour l'apprentissage automatique, tout en identifiant les caractéristiques clés et les comportements typiques des utilisateurs.
- Comparer et sélectionner les techniques de Machine Learning les plus adaptées à la détection des fraudes, en utilisant des méthodes supervisées (classification).
- Traiter le déséquilibre des classes à l'aide de techniques appropriées (rééchantillonnage, ajustement des poids, etc.) pour détecter efficacement les fraudes rares sans générer de faux positifs excessifs.
- Évaluer la performance du modèle en utilisant des métriques adaptées (précision, rappel, F-mesure, AUC-ROC) et optimiser les hyperparamètres afin d'obtenir des résultats fiables et robustes.
- Développer une application web interactive permettant de visualiser les prédictions du modèle, d'analyser les résultats et de soutenir la prise de décision des équipes

de sécurité bancaire.

- Documenter l'ensemble du processus et proposer des recommandations pour l'intégration du système dans un environnement bancaire réel, afin de renforcer la sécurité et la fiabilité des transactions.

2.3 Enjeux et motivations

Le projet de détection des fraudes bancaires soulève des enjeux multiples : techniques, économiques, sociétaux et personnels.

Sur le plan **technique**, il s'agit de tirer parti des capacités du *Machine Learning* pour automatiser l'identification de comportements suspects au sein d'un volume important de données financières. Contrairement aux approches basées sur des règles fixes, les algorithmes d'apprentissage automatique offrent une adaptabilité continue face à l'évolution des techniques de fraude et une meilleure gestion des données déséquilibrées. Cette approche permet non seulement d'accélérer la détection, mais aussi d'améliorer la précision et de réduire le nombre de faux positifs, apportant ainsi une réelle valeur ajoutée par rapport aux méthodes classiques.

Sur le plan **économique**, la fraude bancaire représente des pertes considérables pour les institutions financières, estimées à plusieurs milliards de dollars chaque année. Au-delà de l'impact financier, elle nuit à la confiance des clients et à la réputation des établissements. Le développement d'un système intelligent et proactif de détection constitue donc un enjeu stratégique, combinant rentabilité, sécurité et protection des consommateurs.

Sur le plan **sociétal**, la lutte contre la fraude contribue à renforcer la confiance dans le système financier et à protéger les utilisateurs contre les pertes économiques ou psychologiques liées aux activités frauduleuses. Un système de détection performant participe ainsi à la stabilité économique et renforce la sécurité collective.

Enfin, sur le plan **personnel**, ce projet traduit ma volonté d'approfondir mes compétences en data science et en intelligence artificielle, tout en mobilisant mes connaissances en développement web pour concevoir une solution complète et fonctionnelle. Il m'a permis de maîtriser l'ensemble du cycle de vie d'un projet data — depuis la collecte et le nettoyage des données jusqu'à la mise en production du modèle prédictif et son intégration dans une application web. C'est un projet formateur et enrichissant, à la fois technique et

concret, qui me permet de travailler sur un problème réel à fort impact et de développer une expertise recherchée dans le monde professionnel.

2.4 Méthodologie adoptée

Avant de démarrer la réalisation du projet, j'ai commencé par me former aux bases du *Machine Learning* afin de bien comprendre ses concepts fondamentaux, ses principaux algorithmes et les étapes nécessaires à la construction d'un modèle prédictif. Cette phase d'apprentissage m'a permis d'acquérir une vision globale du cycle de vie d'un projet data, depuis la préparation des données jusqu'à l'évaluation du modèle, tout en me familiarisant avec les outils et bibliothèques utilisés dans ce domaine.

Une fois cette base acquise, j'ai adopté une méthodologie structurée inspirée des bonnes pratiques du développement de projets en data science. La démarche s'est articulée autour de plusieurs étapes complémentaires :

- **Analyse du problème et compréhension du domaine** : J'ai étudié le phénomène de la fraude bancaire afin de cerner ses mécanismes, identifier les différents types de fraudes et déterminer les critères qui rendent une transaction suspecte. Cette phase a permis de préciser les objectifs du projet et de définir les besoins techniques et fonctionnels.
- **Collecte, préparation et exploration des données** : Les données de transactions ont été collectées, nettoyées et mises en forme pour garantir leur qualité. Une analyse exploratoire a ensuite permis de mieux comprendre leur structure, de visualiser les tendances et de repérer les anomalies.
- **Modélisation et entraînement** : Plusieurs algorithmes de *Machine Learning* ont été testés notamment, la régression logistique, les forêts aléatoires ..., afin d'identifier celui offrant le meilleur compromis entre performance et interprétabilité. Des techniques de rééchantillonnage ont été appliquées pour traiter le déséquilibre des classes, fréquent dans les jeux de données liés à la fraude.
- **Évaluation et optimisation du modèle** : Les performances ont été évaluées à l'aide d'indicateurs tels que la précision, le rappel, la F-mesure et la courbe AUC-ROC. Cette phase a permis d'affiner les hyperparamètres et d'améliorer la robustesse du modèle.
- **Déploiement et intégration** : Le modèle final a été intégré dans une application web interactive permettant de visualiser les résultats, d'interpréter les prédictions et d'aider à la prise de décision. Cette interface constitue la partie concrète et opérationnelle du projet.

En suivant cette méthodologie, le projet a permis non seulement d'appliquer les

concepts du *Machine Learning*, mais aussi de comprendre leur mise en œuvre dans un cas réel à fort enjeu, tout en consolidant mes compétences pratiques et analytiques.

Conclusion du chapitre

En somme, ce chapitre a permis de présenter les fondations du projet, en décrivant le contexte dans lequel il s'inscrit, les motivations qui ont conduit à son choix, ainsi que les objectifs visés et la méthodologie suivie pour sa réalisation. Le projet de détection des fraudes bancaires s'appuie sur une approche scientifique rigoureuse, combinant apprentissage automatique et développement web, afin d'apporter une solution innovante à un problème concret. Le chapitre suivant sera consacré à la **phase de conception et de mise en œuvre**, où seront détaillés les outils utilisés, les étapes de développement du modèle de Machine Learning, ainsi que son intégration au sein de l'application web.

3. Conception du système

Introduction

Ce chapitre présente la conception globale du système de détection des fraudes bancaires. Il décrit la manière dont les besoins fonctionnels identifiés précédemment ont été traduits en une architecture claire, modulaire et cohérente. Cette étape a pour objectif de définir la structure logique du système, d'organiser ses différents composants et d'assurer leur interaction harmonieuse. Nous y détaillons les principales couches du système, la modélisation des données ainsi que les technologies retenues pour le développement et l'intégration du modèle de Machine Learning.

3.1 Analyse fonctionnelle

Introduction

L'analyse fonctionnelle vise à définir les fonctionnalités essentielles du système et à préciser les interactions entre les différents acteurs. Elle permet de traduire les besoins exprimés par les utilisateurs en fonctions concrètes avant de passer à la conception technique. Dans le cadre de ce projet, le système a pour but de détecter les fraudes bancaires à l'aide d'un modèle de Machine Learning intégré dans une interface web intuitive et ergonomique.

Identification des acteurs

Le système comporte deux types d'utilisateurs principaux :

- **Utilisateur normal** : peut consulter ses transactions, importer un fichier CSV pour prédire plusieurs transactions, effectuer une prédiction sur une seule transaction via un formulaire, consulter les résultats détaillés (probabilité de fraude, facteurs explicatifs) et accéder à des visualisations interactives.
- **Administrateur** : peut ajouter, supprimer ou mettre à jour les modèles de détection, superviser les transactions analysées, gérer les utilisateurs et assurer la

maintenance générale du système.

Besoins fonctionnels

Les besoins fonctionnels identifiés sont :

- Authentification et gestion des utilisateurs selon leur rôle .
- Importation et prétraitement des données de transactions .
- Exécution du modèle de Machine Learning pour la prédiction des fraudes .
- Génération des facteurs explicatifs pour chaque prédiction .
- Visualisation détaillée des résultats et des facteurs explicatifs .
- Gestion et mise à jour des modèles de détection par l'administrateur .
- Exportation et génération de rapports analytiques.

Cas d'utilisation du système

Les principales interactions entre les acteurs et le système sont illustrées dans le diagramme de cas d'utilisation (figure 3.1).

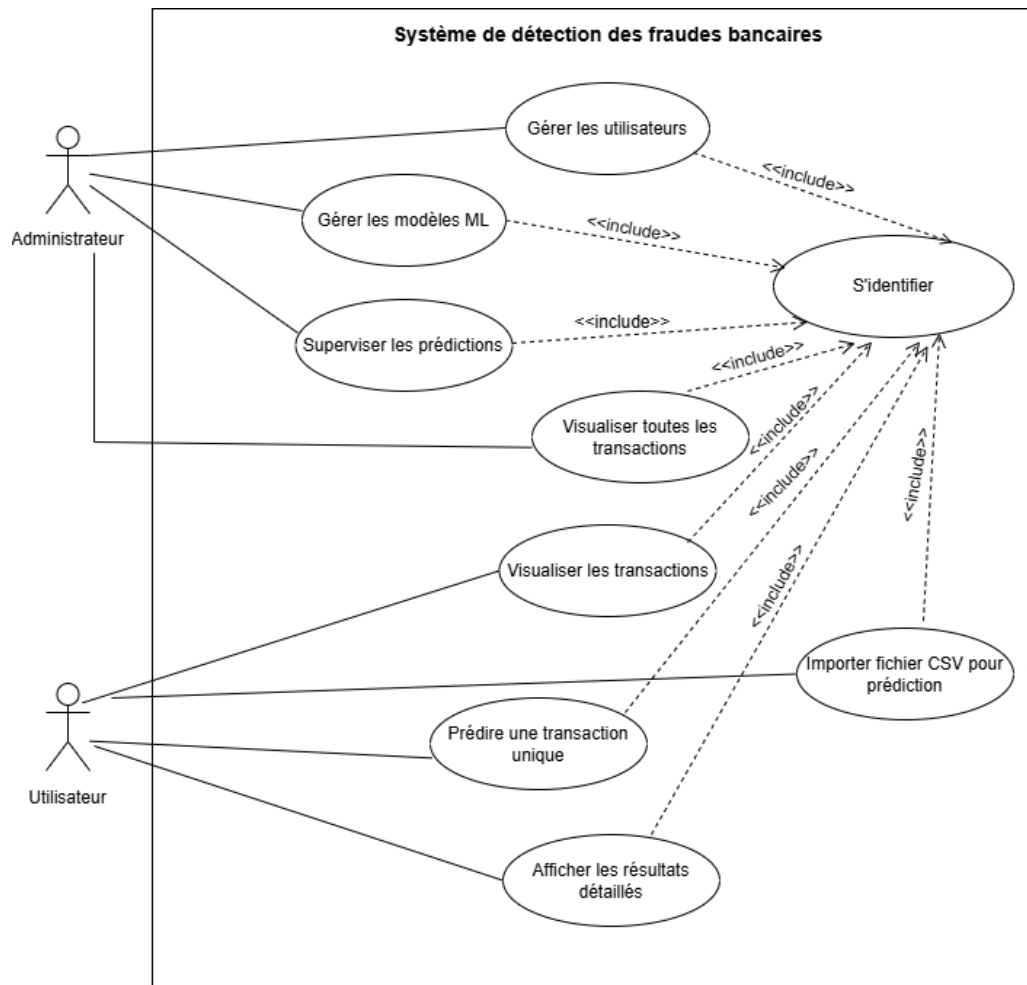


FIGURE 3.1 – Diagramme de cas d'utilisation du système de détection des fraudes bancaires

Description fonctionnelle globale

Pour garantir une conception claire et évolutive, le système est structuré en plusieurs modules interconnectés :

- **Module de gestion des utilisateurs** : gère la connexion, l'inscription et le contrôle des droits d'accès .
- **Module de prédiction** : collecte et traite les transactions, applique le modèle de Machine Learning et retourne le résultat de prédiction .
- **Module de visualisation et reporting** : présente les résultats sous forme de tableaux et graphiques interactifs, avec possibilité d'exporter les rapports .
- **Module d'administration** : permet la gestion des modèles, la supervision des transactions et la maintenance générale du système.

Contraintes fonctionnelles

- Le système accepte uniquement les fichiers CSV conformes au format défini.

- La prédiction d’une transaction doit être effectuée en moins de deux secondes.
- Le modèle ML doit être robuste face au déséquilibre des classes.
- L’interface doit être intuitive et accessible aux utilisateurs non techniques.
- Le système doit prendre en charge l’import de fichiers CSV pouvant atteindre 10 Mo.

Contraintes non-fonctionnelles

Le système respecte les contraintes non-fonctionnelles suivantes :

- **Performance** : Le système assure des temps de réponse optimisés pour les prédictions de fraude.
- **Sécurité** : Protection des données utilisateurs par authentification JWT et hachage sécurisé des mots de passe avec bcrypt.
- **Maintenabilité** : Architecture modulaire à trois couches (Front-end, Back-end, Base de données) facilitant les mises à jour et l’ajout de nouvelles fonctionnalités.
- **Utilisabilité** : Interface web responsive développée avec React.js et TailwindCSS, compatible avec les navigateurs modernes et accessible sur mobile.

Conclusion

Cette analyse fonctionnelle a permis d’identifier les besoins, les acteurs et les fonctionnalités clés du système. Elle constitue la base de la conception technique et servira de référence à la modélisation UML et à la définition de l’architecture générale.

3.2 Architecture générale du projet

Introduction

L’architecture générale décrit la structure du système et la manière dont les composants interagissent pour assurer la détection et la gestion des fraudes. Le projet adopte une approche modulaire et multicouche garantissant la maintenabilité, la sécurité et l’évolutivité.

Architecture modulaire

Le système adopte une **architecture modulaire à trois couches (3-tiers)**, permettant une séparation claire entre la présentation, la logique métier et la gestion des données. Cette structure favorise la maintenabilité, la flexibilité et l’évolutivité du projet.

- **Couche présentation (Front-end)** : développée avec React.js, elle offre une interface web fluide, moderne et conviviale. Cette couche permet à l’utilisateur de

se connecter, d'importer ses fichiers de transactions, d'effectuer des prédictions et de visualiser les résultats sous forme de tableaux, de graphiques et de rapports interactifs. L'interface est responsive et optimisée pour une expérience utilisateur optimale sur différents dispositifs.

- **Couche logique métier (Back-end)** : construite avec FastAPI en Python, elle constitue le cœur du système. Elle gère l'authentification, le prétraitement des données, l'exécution du modèle de Machine Learning, la génération des explications, ainsi que la gestion des utilisateurs, des transactions et des résultats. Cette couche expose également des API REST sécurisées assurant une communication fluide entre le front-end et la base de données.
- **Couche données (Base de données)** : implémentée avec PostgreSQL, elle assure le stockage structuré et sécurisé de l'ensemble des informations du système : comptes utilisateurs, transactions, résultats de prédiction et paramètres des modèles de Machine Learning. Cette couche garantit l'intégrité, la cohérence et la persistance des données tout au long du processus d'analyse.

Communication inter-couches

La communication entre les différentes couches s'effectue via des API REST. Le front-end envoie des requêtes HTTP au back-end pour récupérer ou soumettre des données. Le back-end accède à la base de données via l'ORM SQLAlchemy. Cette architecture modulaire garantit un couplage faible entre les composants et facilite la maintenance du système.

Flux de fonctionnement détaillé

Le fonctionnement global du système suit les étapes suivantes :

1. **Authentification** : Vérification des identifiants utilisateur et génération d'un token JWT pour sécuriser les requêtes ultérieures .
2. **Importation des données** : Validation du format du fichier CSV, parsing et vérification de la conformité des données avec le schéma attendu .
3. **Prétraitement** : Nettoyage des données (gestion des valeurs manquantes, détection des anomalies), normalisation des variables numériques et encodage des variables catégorielles .
4. **Feature engineering** : Extraction et création de nouvelles caractéristiques pertinentes pour améliorer la performance du modèle .
5. **Prédiction** : Chargement du modèle ML entraîné, exécution de la prédiction et calcul des probabilités de fraude .

6. **Explicabilité** : Génération des facteurs explicatifs identifiant les caractéristiques ayant le plus contribué à la décision du modèle .
7. **Persistance** : Sauvegarde des transactions analysées et des résultats de prédiction en base de données pour traçabilité et audit .
8. **Visualisation** : Affichage interactif des résultats avec graphiques, tableaux de bord et possibilité d'export en format PDF ou CSV.

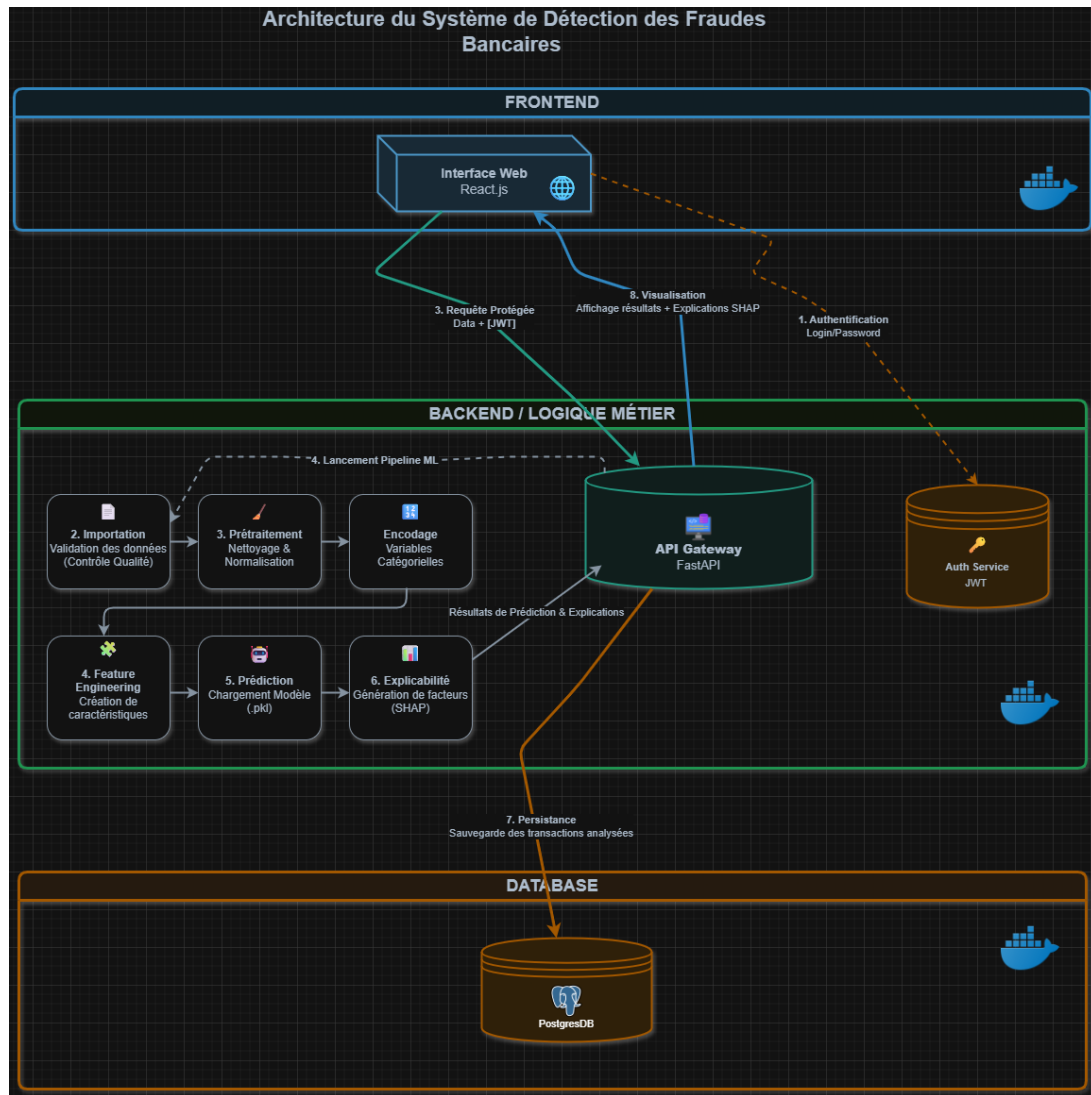


FIGURE 3.2 – Diagramme d'architecture générale du système de détection des fraudes bancaires

3.3 Modélisation des données

Introduction

La modélisation des données définit la structure logique du système. Elle décrit les entités principales, leurs attributs et leurs relations pour garantir la cohérence, l'intégrité

et la traçabilité des informations échangées.

Diagramme de classes

Le diagramme de classes (figure 3.3) présente les entités clés du système avec leurs attributs et méthodes :

- **Utilisateur** : Représente un compte utilisateur avec son rôle (normal/administrateur) .
- **Transaction** : Encapsule les informations d’une transaction bancaire à analyser .
- **RésultatPrédiction** : Stocke les résultats de l’analyse (probabilité de fraude, classification, facteurs explicatifs) .
- **ModèleML** : Représente un modèle de Machine Learning avec ses métadonnées et performances.

Les relations entre ces entités garantissent la traçabilité complète : chaque prédiction est liée à une transaction, un utilisateur et un modèle spécifique.

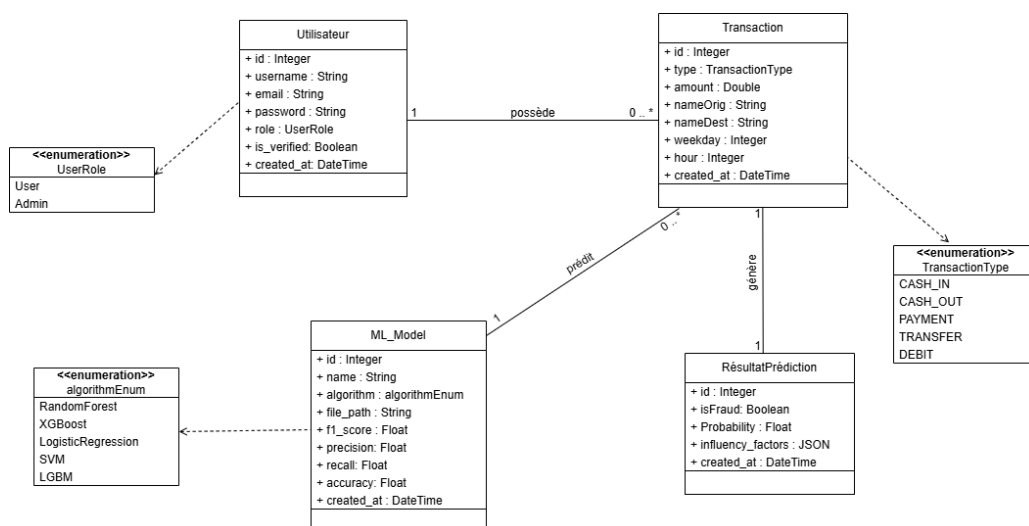


FIGURE 3.3 – Diagramme de classes du système de détection des fraudes bancaires

Modèle de base de données

La structure de la base de données PostgreSQL reflète le diagramme de classes avec les tables principales suivantes :

- **users** : Stocke les informations des utilisateurs (id, username, email, password_hash, role, created_at) .
- **transactions** : Enregistre les transactions analysées avec leurs caractéristiques (id, user_id, amount, merchant, category, timestamp, etc.) .
- **predictions** : Contient les résultats de prédiction (id, transaction_id, model_id, fraud_probability, is_fraud, explanation, predicted_at) .

- **ml_models** : Gère les métadonnées des modèles (id, name, version, accuracy, precision, recall, f1_score, created_at, is_active).

Les contraintes d'intégrité référentielle assurent la cohérence des données, tandis que les index sur les colonnes fréquemment interrogées optimisent les performances des requêtes.

Diagrammes de séquence

Les diagrammes de séquence suivants montrent la dynamique des échanges pour deux cas principaux :

- **Prédiction d'une transaction unique .**
- **Analyse d'un fichier CSV complet.**

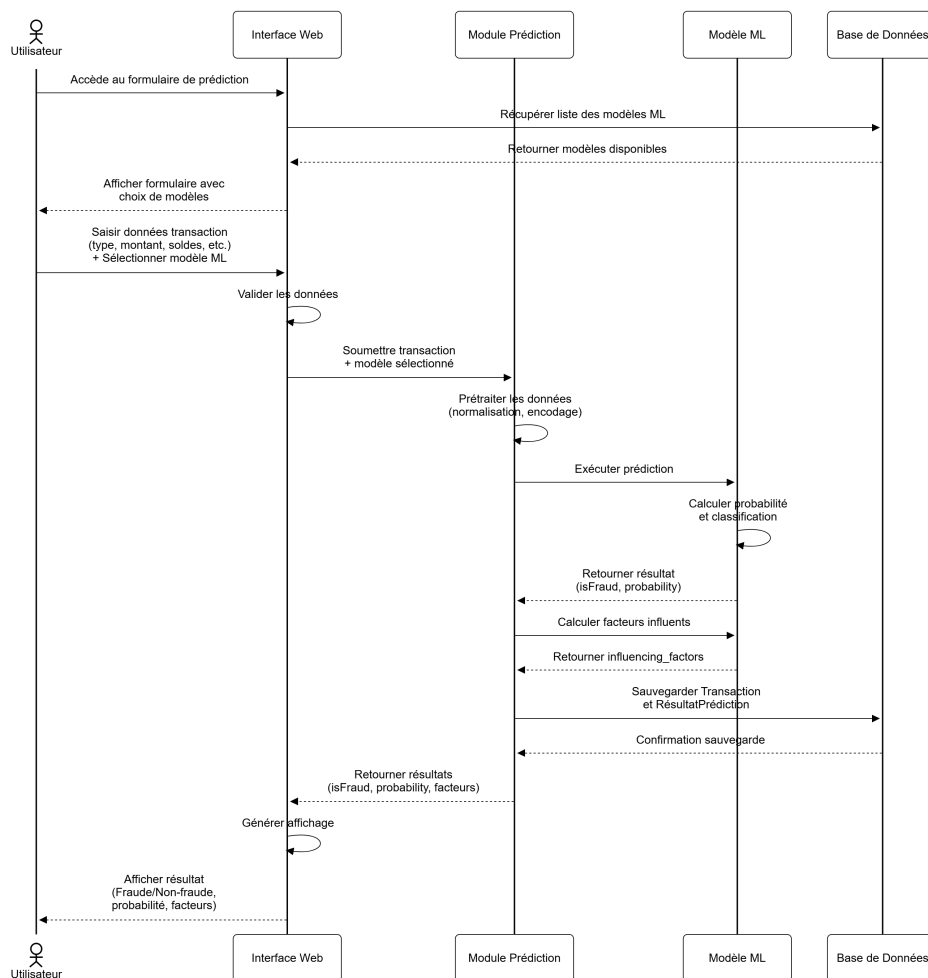


FIGURE 3.4 – Diagramme de séquence du processus de prédiction d'une transaction unique

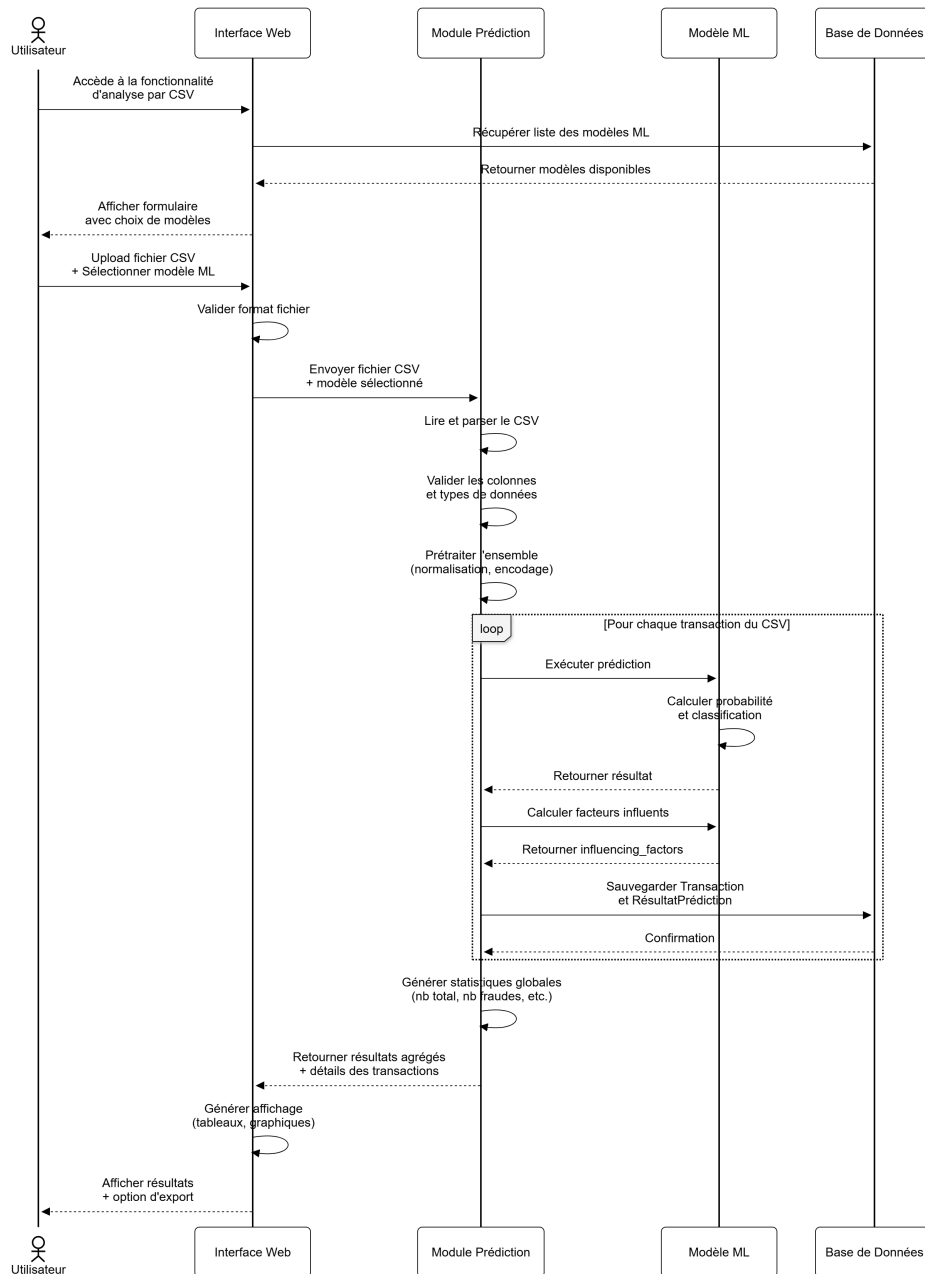


FIGURE 3.5 – Diagramme de séquence du processus d'analyse d'un fichier CSV

3.4 Choix technologiques

Introduction

Les choix technologiques ont été guidés par la recherche d'un équilibre entre performance, maintenabilité, compatibilité et sécurité. Cette section présente les technologies retenues et justifie ces choix.

Technologies et outils utilisés

Le développement du système s'appuie sur un ensemble de technologies et d'outils organisés selon leur fonction dans le projet.

Technologies du système web

Le tableau 3.1 présente les technologies utilisées pour le développement de l'application web.

Couche	Technologie	Rôle principal
Front-end	React.js, TailwindCSS, Chart.js	Interface utilisateur responsive avec visualisations interactives.
Back-end	FastAPI, SQLAlchemy, Pydantic	API REST, validation des données et communication avec la base de données.
Base de données	PostgreSQL	Stockage structuré des utilisateurs, transactions et résultats de prédiction.
Sécurité	JWT, bcrypt	Authentification des utilisateurs et chiffrement des mots de passe.
Gestion de version	Git, GitHub	Gestion de version, Suivi des versions et visualisation de l'avancement du projet par l'encadrant.

TABLE 3.1 – Technologies utilisées pour le système web

Outils de Machine Learning

Bibliothèque	Utilisation
Scikit-learn	Entraînement et évaluation des modèles de classification.
Pandas	Manipulation et prétraitement des données.
NumPy	Calculs numériques et opérations matricielles.
Joblib	Sauvegarde et chargement du modèle entraîné.

TABLE 3.2 – Bibliothèques de Machine Learning

Outils d'analyse et de visualisation

Durant la phase d'exploration des données et d'entraînement du modèle, plusieurs outils de visualisation ont été utilisés dans des notebooks Jupyter :

- **Seaborn** : Visualisation des distributions, matrices de corrélation et analyses statistiques avancées.
- **Plotly** : Génération de graphiques interactifs pour l'exploration approfondie des données.

Ces outils ont permis d'analyser les caractéristiques des transactions frauduleuses, d'identifier les corrélations entre variables et d'évaluer visuellement les performances des différents modèles testés.

Outils de déploiement et conteneurisation

- **Docker** : Conteneurisation des composants du système (front-end, back-end, base de données) pour garantir la portabilité et faciliter le déploiement.
- **Docker Compose** : Orchestration des conteneurs et gestion des dépendances entre services.

L'utilisation de Docker permet d'assurer que le système fonctionne de manière identique sur différents environnements (développement, test, production) et simplifie le processus de déploiement.

Justification des choix technologiques

Front-end : React.js

- Architecture basée sur des composants réutilisables facilitant la maintenance et l'organisation du code.
- Écosystème riche avec de nombreuses bibliothèques complémentaires (Chart.js pour les visualisations, Axios pour les requêtes HTTP).
- Large communauté et documentation extensive facilitant le développement.
- Facilité de création d'interfaces interactives et dynamiques.

Back-end : FastAPI

- Framework Python moderne offrant de bonnes performances pour les API REST.
- Documentation automatique (Swagger/OpenAPI) facilitant les tests et l'intégration.
- Validation automatique des données avec Pydantic réduisant les erreurs.
- Syntaxe Python claire avec support des annotations de type.
- Excellente compatibilité avec les bibliothèques de Machine Learning (Scikit-learn, Pandas, NumPy).

Base de données : PostgreSQL

- Base de données relationnelle robuste et fiable pour le stockage structuré des données.

- Support des contraintes d'intégrité référentielle garantissant la cohérence des données.
- Compatibilité native avec SQLAlchemy facilitant l'intégration avec Python.
- Open-source avec une documentation complète et une large communauté.
- Adapté au stockage des utilisateurs, transactions et résultats de prédiction.

Machine Learning : Scikit-learn

- Bibliothèque mature et stable avec implémentations optimisées des algorithmes classiques.
- API cohérente facilitant l'expérimentation de différents modèles.
- Documentation complète et nombreux tutoriels disponibles.
- Intégration native avec NumPy et Pandas pour le traitement des données.
- Support de la sérialisation des modèles avec Joblib pour le déploiement.

Gestion de version : Git et GitHub

- Git permet un suivi précis de l'évolution du code, des fonctionnalités et des corrections de bugs.
- GitHub sert de dépôt distant pour sauvegarder le projet et permettre à l'encadrant de suivre l'avancement.
- Facilite la gestion des versions et la récupération des états précédents du projet si nécessaire.
- Permet également de documenter le projet via le README et les commits clairs.

Conclusion

Les choix technologiques retenus assurent la robustesse, la performance et l'évolutivité du système. Ils offrent une base solide pour l'intégration future de nouveaux modèles de Machine Learning et garantissent une maintenance facilitée.

Conclusion du chapitre

La conception du système a permis de définir son organisation interne, son architecture logicielle et ses principales composantes. L'adoption d'une architecture modulaire à trois couches favorise la maintenabilité et l'évolutivité du projet, tout en garantissant une séparation claire entre les responsabilités. Les choix technologiques effectués offrent un équilibre entre performance, simplicité de développement et compatibilité avec le modèle de Machine Learning. Cette phase de conception constitue une base solide pour la mise en œuvre du système, détaillée dans le chapitre suivant consacré au développement et à l'intégration des modules.

4. Réalisation et mise en œuvre

Introduction

Ce chapitre décrit le processus de réalisation du système de détection des fraudes bancaires, depuis le développement de l'application web jusqu'à l'intégration du modèle de Machine Learning. L'objectif est de présenter les différentes étapes techniques mises en œuvre pour concrétiser l'architecture logicielle définie au chapitre précédent.

La mise en œuvre s'articule autour de plusieurs axes :

- Le **développement web**, comprenant la conception de l'interface utilisateur et la création de l'API REST assurant la communication avec la base de données .
- Le **pipeline de traitement des données**, responsable de la préparation, de la normalisation et de la validation des entrées avant la prédiction .
- La **création et l'entraînement du modèle de Machine Learning**, incluant le choix de l'algorithme, la gestion des jeux de données et l'évaluation des performances .
- Le **déploiement et l'intégration du système**, garantissant une exécution fiable et homogène sur différents environnements grâce à Docker.

Cette phase met en évidence la transition entre la conception théorique et la mise en œuvre concrète du système, illustrée à travers des interfaces, des extraits de code et des tests fonctionnels. Le lecteur pourra ainsi visualiser la cohérence de l'architecture et la modularité des composants développés.

4.1 Développement web (interface et API)

Introduction

Le développement web constitue la première étape de la réalisation du système. Il a pour objectif de fournir une interface utilisateur ergonomique et intuitive ainsi qu'une API performante permettant la gestion sécurisée des données et des échanges entre les différentes composantes du système.

Le développement s'appuie sur une architecture **client-serveur**, où :

- le **Front-end (React.js + TailwindCSS)** gère la présentation, la navigation et les interactions avec l'utilisateur, choisi pour sa modularité, sa réactivité et sa grande communauté .
- le **Back-end (FastAPI)** gère les requêtes, la logique métier et la communication avec la base de données PostgreSQL, choisi pour sa rapidité et sa compatibilité avec les bibliothèques de Machine Learning.

Cette organisation permet de séparer clairement les responsabilités et d'assurer une maintenance aisée tout en garantissant une expérience utilisateur fluide et réactive.

4.1.1 Développement de l'interface utilisateur (Front-end)

L'interface utilisateur a été développée avec **React.js** et **TailwindCSS**, offrant une conception moderne, réactive et modulaire. L'objectif est de permettre à l'utilisateur de naviguer facilement entre les fonctionnalités : consultation des transactions, prédiction des fraudes et visualisation des résultats.

Page d'accueil (HomePage)

Avant l'authentification, l'utilisateur est accueilli par une page d'introduction présentant le système de détection des fraudes, avec des boutons pour se connecter ou créer un compte.

Fonctionnalités principales :

- Présentation générale du système et des services proposés .
- Accès rapide aux pages de connexion et d'inscription .
- Design responsive et attractif pour une première impression engageante.



FIGURE 4.1 – Page d’accueil présentant le système avant authentification, offrant un accès rapide aux principales fonctionnalités

Fonctionnalités principales du Front-end

- Authentification sécurisée avec JWT et stockage côté client (LocalStorage) .
- Tableau de bord interactif affichant graphiques et statistiques en temps réel (Chart.js) .
- Formulaire d’importation CSV et visualisation dynamique des résultats .
- Section d’administration pour la supervision des modèles et la gestion des utilisateurs.

Structure générale du Front-end

La structure du Front-end est organisée pour séparer clairement les responsabilités et faciliter la maintenance. Chaque dossier correspond à un rôle spécifique : composants, pages, routes, contexte utilisateur, hooks et API.

```
src/
|
|  api/                # Appels API via Axios avec injection du JWT
|  context/           # Gestion du contexte utilisateur (JWT, session)
|  hooks/              # Hooks personnalisés
|  layouts/           # Layouts génériques (DashboardLayout, AuthLayout)
|  pages/
|    admin/           # Pages réservées à l'administration
|    guest/           # Pages accessibles sans connexion
|    user/            # Pages utilisateurs connectés
```

```

|    NoFoundPage.js
|
|    routes/
|
|    AppRoutes.js
|
|    ProtectedRoutes.js # Vérification de l'existence du JWT
App.jsx                # Point d'entrée de l'application React

```

```

dashboard.jsx
frontend-app > src > pages > admin > dashboard.jsx > ProfessionalDashboard
131 export default function ProfessionalDashboard() {
140 // Appel à l'API pour récupérer les statistiques des transactions
141 useEffect(() => {
142   const fetchStats = async () => {
143     try {
144       setLoading(true);
145       const response = await axiosClient.get("/transactions/stats/yearly");
146       setApiData(response.data);
147       setError(null);
148     } catch (err) {
149       console.error("Erreur lors de la récupération des données:", err);
150       setError("Impossible de charger les données. Affichage des données de démonstration.");
151       // On continue avec les données simulées en cas d'erreur
152     } finally {
153       setLoading(false);
154     }
155   };
156   fetchStats();
157 }, []);
158
159 // Appel à l'API pour récupérer les statistiques des modèles ML
160 useEffect(() => {
161   const fetchMLModelsStats = async () => {
162     try {
163       setMLModelsLoading(true);
164       const response = await axiosClient.get("/ml-models/stats");
165       setMLModelsData(response.data);
166       setMLModelsError(null);
167     } catch (err) {
168

```

FIGURE 4.2 – Exemple de composant React : Dashboard.jsx illustrant la modularité et la lisibilité du code front-end

Captures d'écran des interfaces principales

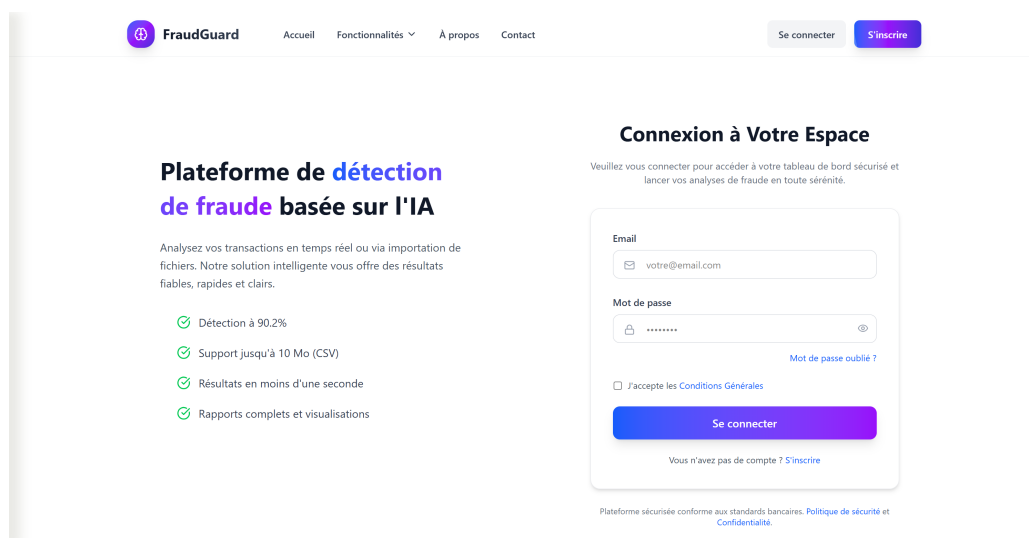


FIGURE 4.3 – Interface de connexion sécurisée avec JWT, offrant une expérience utilisateur claire et intuitive

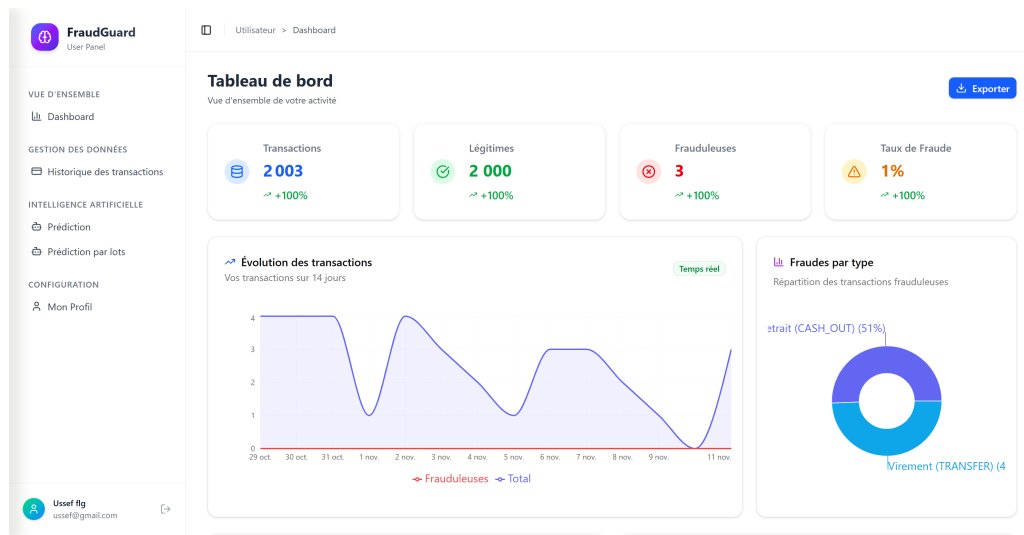


FIGURE 4.4 – Tableau de bord principal affichant les statistiques de détection en temps réel, conçu pour la lisibilité et l’interactivité

The 'Analyse de Fichier CSV' page is designed for users to upload a CSV file for fraud analysis. It includes a section for 'Analyse par Lot' where users can upload a CSV file. Below this, there is a 'Modèle d'analyse' dropdown menu and a 'Fichier CSV' upload area with a large blue button labeled 'Analyser le fichier'. The page also includes a 'Conseils d'Upload' section at the bottom.

FIGURE 4.5 – Page de prédiction des fraudes via formulaire ou fichier CSV, offrant plusieurs options d’entrée pour l’utilisateur

FraudGuard

User Panel

VUE D'ENSEMBLE

Dashboard

GESTION DES DONNÉES

Historique des transactions

INTELLIGENCE ARTIFICIELLE

Prediction

Prediction par lots

CONFIGURATION

Mon Profil

Utilisateur > Prediction

Détection de Fraude Intelligente

Système avancé d'analyse transactionnelle utilisant l'intelligence artificielle.

Analyse de Transaction

Renseignez les détails de la transaction à analyser

Modèle d'analyse

FraudDetection XGB

Precision: 91.6%

Informations de transaction

Type de transaction *

Montant (€) *

Date exacte

Heure

Minute

Informations personnelles

Compte origine *

Compte destination *

Soldes des comptes

Ancien solde origine (€)

Nouveau solde origine (€)

Ancien solde destination (€)

Nouveau solde destination (€)

Analyser la transaction

Signes d'alerte

Transactions à des heures inhabituelles

Montants anormalement élevés

Demandes pressantes

Useef fig

useef@gmail.com

FIGURE 4.6 – Formulaire manuel pour la prédiction des fraudes, facilitant l'entrée individuelle de transactions

FraudGuard

User Panel

VUE D'ENSEMBLE

Dashboard

GESTION DES DONNÉES

Historique des transactions

INTELLIGENCE ARTIFICIELLE

Prediction

Prediction par lots

CONFIGURATION

Mon Profil

Résultats de l'Analyse

Résumé des transactions analysées

FraudDetection XGB

Total Transactions

TOTAL

100

Transactions Légitimes

SÉCURISÉ

100

100.0%

Transactions Frauduleuses

ALERTE

0

0%

Répartition des transactions

100 transactions

0 fraudes 100 legitimes

Niveau de risque

Niveau de Risque Global

Faible Moyen Elevé

Faible

0% de transactions suspectes

Taux de fraude dans les normes

Vos transactions présentent un profil de risque acceptable.

ANALYSES DÉTAILLÉES

Données basées sur l'analyse du fichier

Types de transactions

Répartition par catégorie opérationnelle

DEBIT

TRANSFER

CASH_IN

PAYMENT

CASH_OUT

26

21

19

18

16

Plages de montants

Distribution des valeurs transactionnelles

0-1k

1k-5k

5k-10k

10k-50k

50k+

1

6

8

85

0

Évolution temporelle

Fluctuation des transactions dans le temps

FIGURE 4.7 – Résultats des prédictions à partir d'un fichier CSV, présentés de manière claire et structurée

39

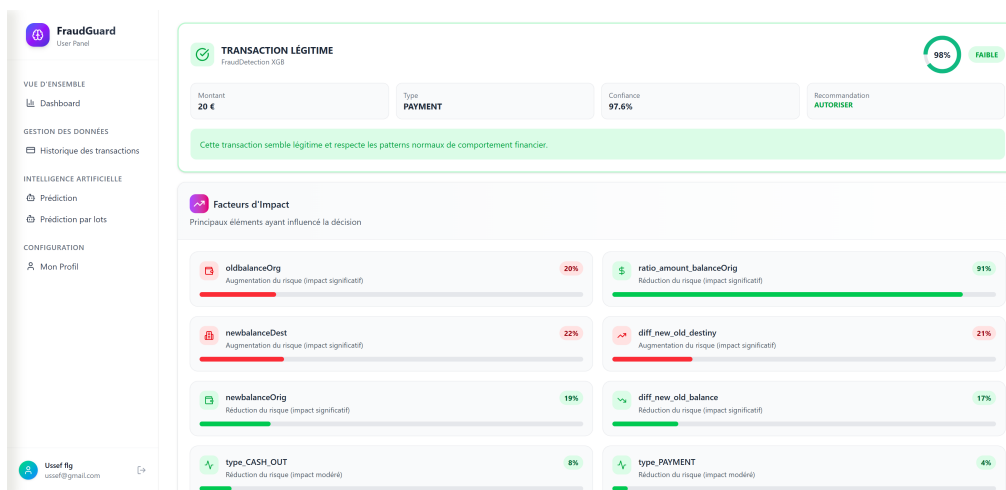


FIGURE 4.8 – Résultats des prédictions via formulaire manuel, avec mise en évidence des transactions suspectes

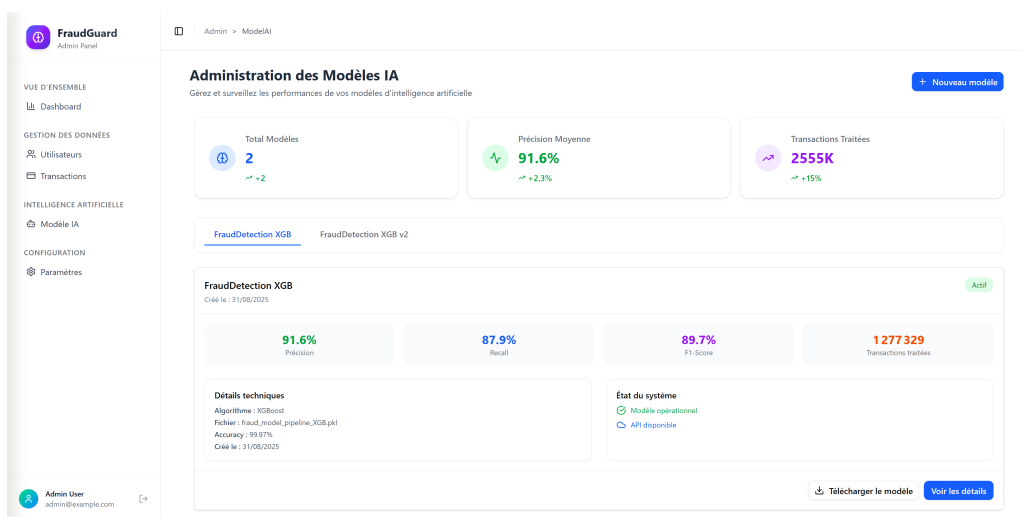


FIGURE 4.9 – Page d'administration pour la gestion des modèles et utilisateurs, offrant un contrôle complet du système

Conclusion

Cette section illustre la modularité, l'ergonomie et la fluidité du Front-end. Les captures d'écran et le code d'exemple montrent la structure claire des composants et l'expérience utilisateur efficace, garantissant une interaction cohérente avec le Back-end et les fonctionnalités du système.

4.1.2 Développement de l'API et du Back-end

Le **Back-end** a été développé avec **FastAPI**, choisi pour sa performance, sa simplicité et sa compatibilité avec les bibliothèques de Machine Learning. Il agit comme une

passerelle entre le Front-end, la base de données et le moteur prédictif, assurant sécurité, cohérence et rapidité.

Sécurité et validation des données

La sécurité et l'intégrité des données sont assurées par :

- **Hashage des mots de passe** via `passlib` et l'algorithme **Bcrypt**, garantissant qu'aucun mot de passe n'est stocké en clair .
- **Validation des schémas JSON** grâce à **Pydantic** (`schemas.py`), assurant la conformité et l'intégrité des données reçues.

Intégration du modèle ML

Le modèle entraîné (`.pkl`) est chargé une seule fois au démarrage de FastAPI pour optimiser les performances. L'API expose un endpoint `/predict` qui reçoit les transactions, les transmet au service de prétraitement, appelle le modèle et renvoie le résultat au Front-end.

Structure modulaire du Back-end

Chaque module (`auth`, `user`, `transactions`, `predictions`, `admin`) possède ses propres fichiers `routes.py`, `services.py`, `models.py` et `schemas.py`. Les dossiers `core` et `utils` contiennent respectivement la configuration globale, la sécurité et les fonctions utilitaires réutilisables.

```
app/
  auth/          # Authentification et gestion JWT
  user/          # Gestion des utilisateurs
  transactions/  # Gestion des transactions CRUD
  predictions/   # Prédiction de fraudes
  admin/         # Administration des modèles et utilisateurs
  core/          # Configuration globale et sécurité
  utils/         # Fonctions utilitaires
  main.py        # Point d'entrée de l'application
  database.py    # Connexion à PostgreSQL
```

Avantages :

- Modularité et maintenance facilitées .
- Séparation claire des responsabilités .
- Cohérence et communication fluide avec le Front-end via REST .

— Évolutivité pour ajouter facilement de nouveaux modules.

Captures d'écran illustratives

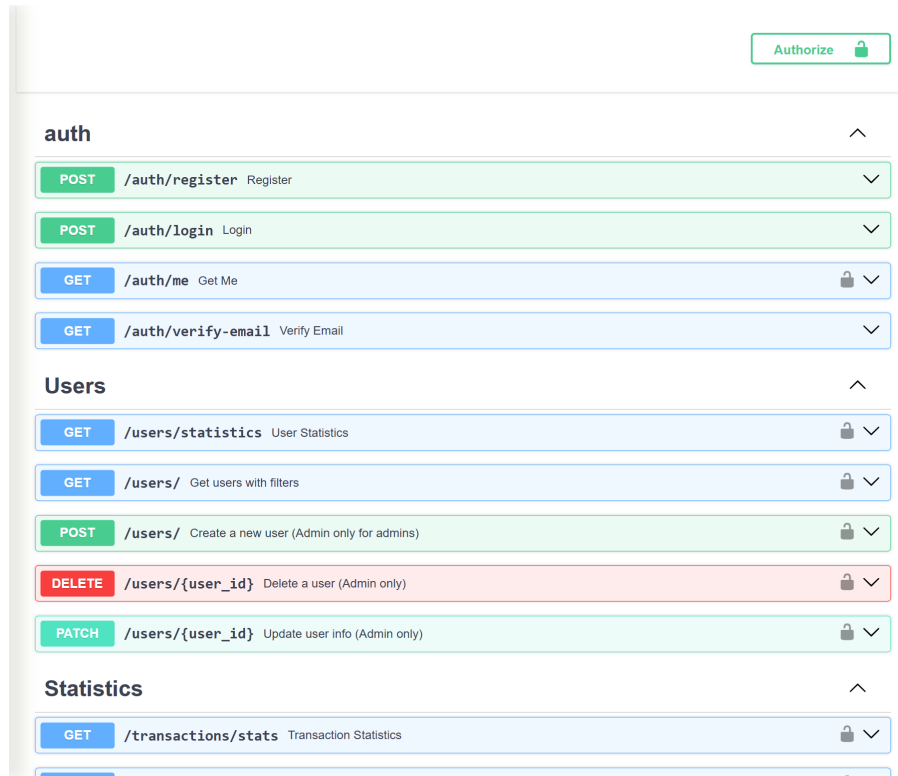


FIGURE 4.10 – Documentation automatique des endpoints via Swagger, facilitant la compréhension et les tests

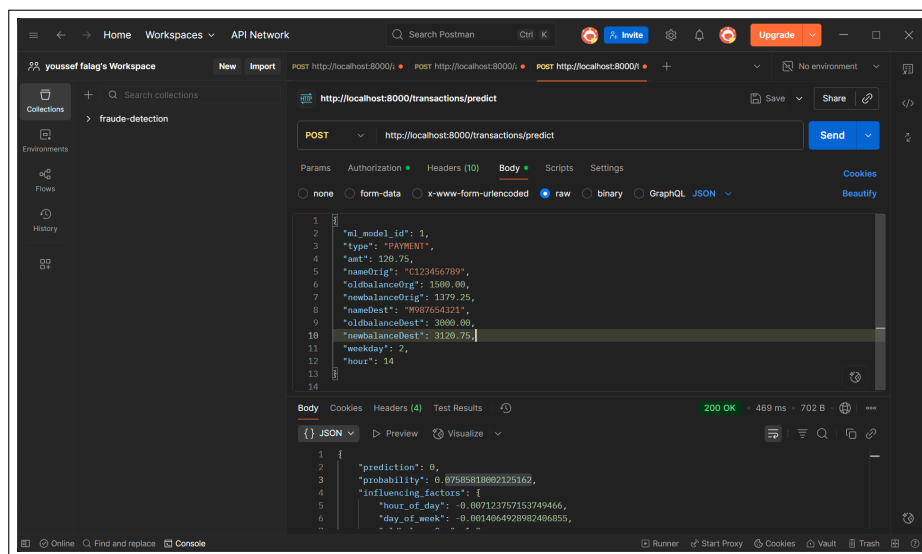


FIGURE 4.11 – Test de l'API de prédiction à l'aide de Postman pour vérifier les réponses et la conformité des endpoints

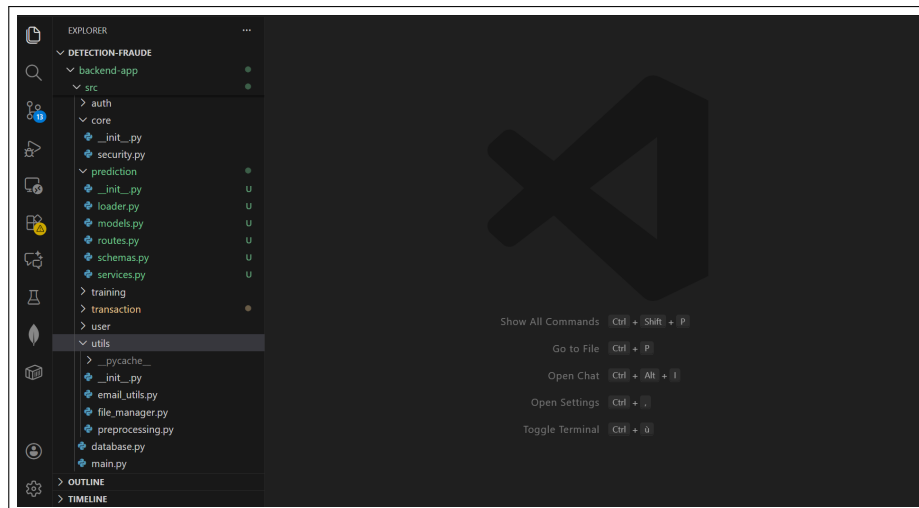


FIGURE 4.12 – Structure générale du projet Back-end FastAPI, illustrant la modularité et la lisibilité du code

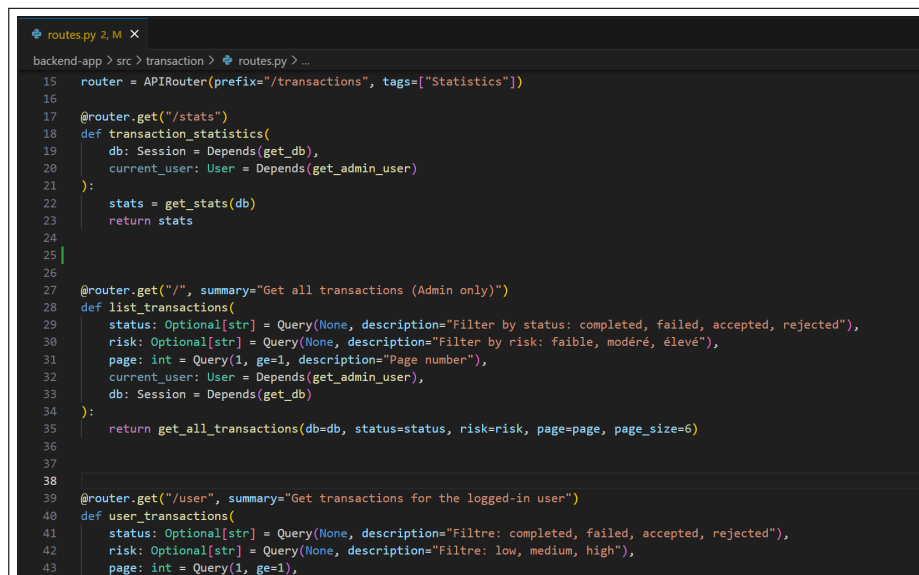


FIGURE 4.13 – Exemple de code : routes du module `transactions`, démontrant la séparation des responsabilités

```

services.py 4 X
backend-app > src > transaction > services.py > ...
737 def get_user_transaction_stats(db: Session, user_id: int):
738     current_year = datetime.now().year
739     prev_year = current_year - 1
740
741     # --- Transactions totales ---
742     total_current = db.query(func.count(Transaction.id)).filter(
743         Transaction.user_id == user_id,
744         func.extract("year", Transaction.created_at) == current_year
745     ).scalar()
746
747     total_prev = db.query(func.count(Transaction.id)).filter(
748         Transaction.user_id == user_id,
749         func.extract("year", Transaction.created_at) == prev_year
750     ).scalar()
751
752     # --- Transactions non frauduleuses ---
753     nonfraud_current = db.query(func.count(Transaction.id)).filter(
754         Transaction.user_id == user_id,
755         func.extract("year", Transaction.created_at) == current_year,
756         Transaction.isFraud == 0
757     ).scalar()
758
759     nonfraud_prev = db.query(func.count(Transaction.id)).filter(
760         Transaction.user_id == user_id,
761         func.extract("year", Transaction.created_at) == prev_year,
762         Transaction.isFraud == 0
763     ).scalar()
764
765     # --- Transactions frauduleuses ---

```

FIGURE 4.14 – Exemple de code : services du module `transactions`, illustrant la logique métier centralisée

4.1.3 Communication Front-end / Back-end

Les échanges entre le Front-end et le Back-end se font via des requêtes **HTTP RESTful** au format **JSON**, avec authentification **JWT**.

Exemple de flux :

1. L'utilisateur se connecte via le formulaire React → POST `/auth/login`.
2. Le serveur valide les identifiants, génère un JWT et le renvoie au client.
3. Le Front-end stocke le JWT et l'envoie pour toutes les requêtes suivantes (`/transactions`, `/predict`).
4. Le Back-end vérifie le JWT, exécute la logique métier et renvoie le résultat au Front-end.

Avantages :

- Séparation nette entre interface et logique métier .
- Sécurité renforcée avec JWT et Bcrypt .
- Performance accrue grâce au chargement unique du modèle ML .
- Modularité facilitant l'évolution du système.

Conclusion

Le développement du Back-end constitue le socle du système. La structure modulaire et l'intégration du modèle de Machine Learning assurent une communication fluide avec le

Front-end et une maintenance aisée. Les captures d'écran et les extraits de code illustrent le bon fonctionnement et la cohérence globale des composants.

Cette section montre également la fluidité et l'ergonomie de l'ensemble Front-end / Back-end, garantissant une expérience utilisateur efficace et une base solide pour le traitement et la prédiction des fraudes.

4.2 Exploration et Présentation des Données

Introduction

Avant de construire notre système de détection de fraudes, il était essentiel de bien comprendre les données avec lesquelles nous allions travailler. On ne peut pas proposer de solution efficace sans d'abord bien connaître le problème.

Cette phase d'exploration des données (qu'on appelle en anglais *Exploratory Data Analysis* — *EDA*) nous a permis de :

- Comprendre la structure générale du dataset
- Détecter les anomalies et valeurs aberrantes
- Observer le déséquilibre entre transactions légitimes et frauduleuses
- Identifier les variables les plus pertinentes pour notre modèle

Les découvertes faites durant cette exploration ont directement guidé nos choix pour le prétraitement des données et la construction des modèles.

4.2.1 Présentation du jeu de données

Nous avons travaillé avec un sous-ensemble simulé du dataset *Credit Card Fraud Detection* disponible sur Kaggle. Ce jeu de données comprend au total **50 000 transactions**, chacune représentant une opération financière avec les informations suivantes :

- **Type de transaction** : CASH_OUT (retrait), TRANSFER (virement), PAYMENT (paiement), DEBIT (débit), CASH_IN (dépôt)
- **Montants et soldes** : montant de la transaction, solde avant et après l'opération pour l'expéditeur et le destinataire
- **Identifiants** : codes anonymisés pour l'expéditeur et le destinataire
- **Variable cible** : `isFraud`, indiquant si la transaction est frauduleuse (1) ou non (0)

Un point important à souligner : seulement **1,29%** des transactions sont frauduleuses. Autrement dit, sur les 50 000 transactions, environ 645 représentent des fraudes. Ce déséquilibre constitue un défi majeur pour l'apprentissage des modèles.

TABLE 4.1 – Aperçu des cinq premières transactions du dataset

Idx	Type	Montant	Exp	Solde Avant	Solde Après	Dest	Solde D. Avant	Solde D. Après	Fraud	Jour	H	Min
0	TRANSFER	1569.07	C90762	2757.79	1188.72	D30885	12.08	1581.15	0	3	12	29
1	CASH_IN	7811.37	C95490	15006.03	7194.66	D59577	2213.39	10024.76	0	5	8	11
2	CASH_IN	8662.14	C40941	11368.42	2706.28	D36222	2288.38	10950.52	0	2	17	47
3	PAYMENT	828.72	C82285	2282.10	1453.38	D56814	0.00	0.00	0	5	19	27
4	DEBIT	4683.77	C28605	12456.40	7772.63	D59857	0.00	0.00	0	1	3	31

Ce tableau offre un aperçu global du dataset, mettant en avant les principales caractéristiques avant d’aborder des analyses plus détaillées telles que la distribution des transactions et les corrélations entre variables.

4.2.2 Analyse statistique globale

Pour débiter notre exploration, nous avons calculé des statistiques descriptives sur l’ensemble du dataset, fournissant des indicateurs tels que la moyenne, l’écart-type, les valeurs minimales et maximales, ainsi que les quartiles.

TABLE 4.2 – Statistiques descriptives des variables numériques du dataset

	amt	oldbalanceOrg	newbalanceOrig	oldbalanceDest	newbalanceDest	isFraud	weekday	hour	minute
count	50000	50000	50000	50000	50000	50000	50000	50000	50000
mean	5004.50	10012.48	5007.98	991.80	2980.16	0.0129	2.82	11.49	29.63
std	2882.08	6647.03	4396.48	1519.68	4185.56	0.1128	1.89	6.91	17.29
min	10.08	11.08	0.05	0.00	0.00	0.00	0.00	0.00	0.00
25%	2518.02	4593.91	1376.01	0.00	0.00	0.00	1.00	6.00	15.00
50%	4983.31	9129.79	3744.54	0.00	0.00	0.00	3.00	11.00	30.00
75%	7508.64	14455.95	7673.52	1868.81	6206.30	0.00	4.00	17.00	45.00
max	9999.83	29907.41	19930.62	4999.61	14929.94	1.00	6.00	23.00	59.00

Distribution des montants

Le tableau montre que les montants présentent une grande variation, avec une moyenne d’environ 5 000 et une dispersion élevée. Les valeurs vont de montants très faibles jusqu’à près de 10 000, ce qui traduit une distribution très étendue. Cette hétérogénéité justifie l’application d’une normalisation avant l’entraînement du modèle.

Variabilité des soldes

Les soldes initiaux et finaux des comptes affichent également une forte dispersion. Les moyennes sont relativement élevées, mais les écarts-types importants indiquent des différences marquées entre les transactions. Certains soldes sont nuls, tandis que d’autres atteignent des valeurs élevées, ce qui renforce la nécessité d’un prétraitement adapté.

Valeurs manquantes

Les statistiques du tableau confirment que l'ensemble des variables contient 50 000 observations sans valeurs manquantes. Les quelques zéros observés dans les soldes destinataires représentent des situations normales pour certaines transactions, et non de réelles données manquantes.

4.2.3 Analyse du déséquilibre des classes

Le déséquilibre entre transactions légitimes et frauduleuses constitue l'un des principaux défis de ce projet.

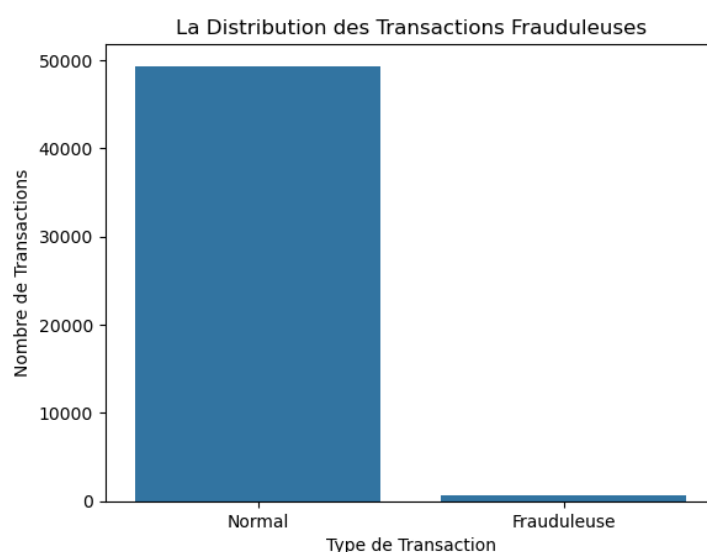


FIGURE 4.15 – Répartition des transactions selon la classe `isFraud`.

La figure ci-dessus illustre le déséquilibre entre les transactions légitimes et frauduleuses. On observe que la grande majorité des transactions sont légitimes, tandis que les fraudes représentent une part extrêmement faible. Cette distribution met en évidence le défi majeur pour l'apprentissage des modèles, qui doivent être capables de détecter les rares fraudes malgré leur faible occurrence.

Sur les 50 000 transactions du dataset :

- **49 355 sont légitimes** (98,71%)
- **645 sont frauduleuses** (1,29%)

Ce ratio de 1 :77 illustre une disproportion extrême qui complique considérablement l'apprentissage du modèle. En effet, sans traitement approprié, un algorithme aurait tendance à favoriser systématiquement la classe majoritaire pour maximiser son taux de réussite apparent, au détriment de la détection des fraudes qui est pourtant notre objectif principal.

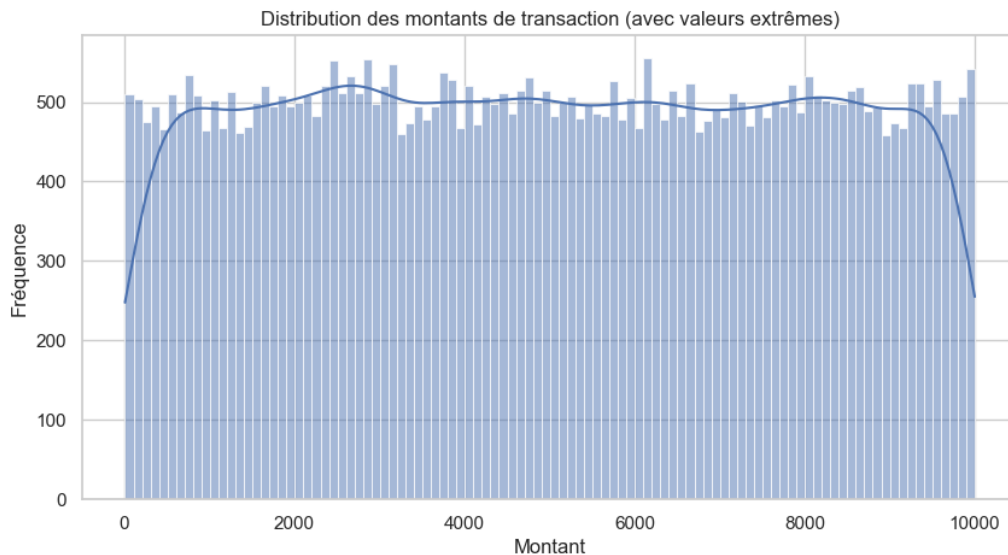


FIGURE 4.16 – Distribution des montants de transaction.

La figure illustre la répartition des montants des transactions. Comme les données sont simulées, la distribution apparaît assez uniforme. Cela permet simplement d’avoir une vue générale des différents niveaux de montants présents dans le dataset.

4.2.4 Analyse par type de transaction

Toutes les transactions ne sont pas égales face au risque de fraude. Nous avons calculé le taux de fraude pour chaque type de transaction afin d’identifier les catégories les plus à risque.

Les résultats sont révélateurs :

- **TRANSFER (virements)** : taux de fraude le plus élevé, autour de 6-7% C’est de loin le type de transaction le plus risqué. Les fraudeurs privilégient les virements car ils permettent de transférer rapidement de l’argent vers d’autres comptes.
- **CASH_OUT (retraits)** : taux de fraude significatif, environ 1-2% Les retraits en espèces sont également ciblés par les fraudeurs car ils permettent de récupérer l’argent de manière relativement anonyme.
- **PAYMENT, DEBIT, CASH_IN** : taux de fraude quasi nuls ($< 0,01\%$) Ces types de transactions sont très rarement frauduleux. Les paiements et dépôts ne présentent pas d’intérêt pour les fraudeurs.

Cette découverte est cruciale : elle nous indique que notre modèle devra être particulièrement vigilant avec les transactions de type TRANSFER et CASH_OUT.

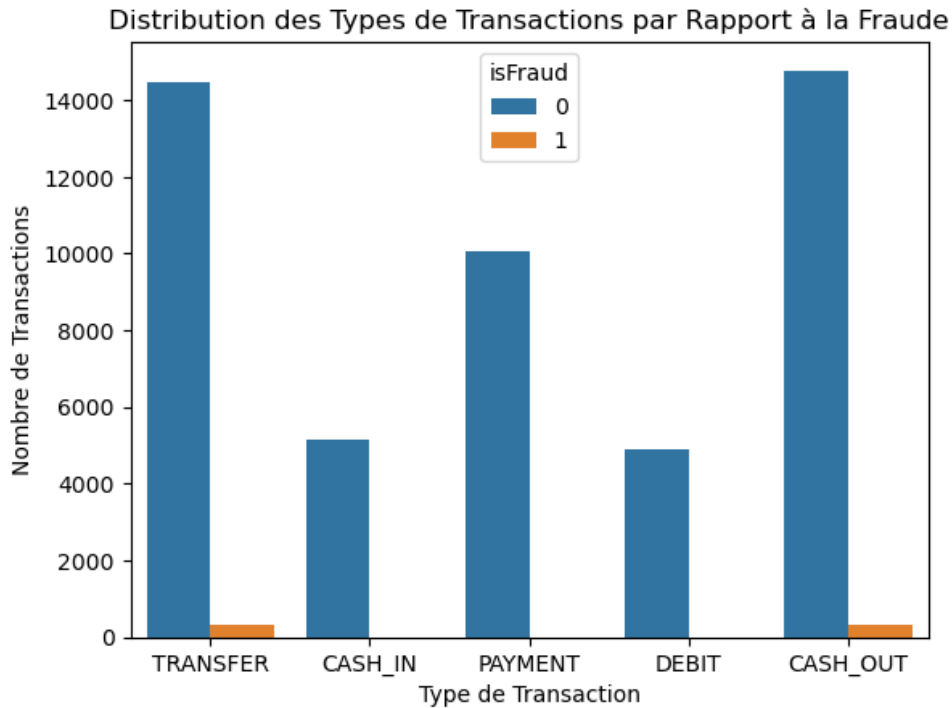


FIGURE 4.17 – Taux de fraude par type de transaction, montrant la concentration sur TRANSFER et CASH_OUT.

Ce graphique à barres met en évidence la concentration du risque de fraude sur deux catégories : TRANSFER (6-7%) et CASH_OUT (1-2%), tandis que les autres types présentent des taux quasi nuls.

4.2.5 Analyse des corrélations entre variables

Pour mieux comprendre les relations entre les différentes variables, nous avons calculé une matrice de corrélation. Celle-ci permet d'identifier quelles variables évoluent ensemble et de repérer d'éventuelles redondances.

Les principaux constats sont les suivants :

- **Forte corrélation** entre `oldBalanceOrig` et `newBalanceOrig` : le solde après transaction dépend logiquement du solde initial.
- **Forte corrélation** entre `oldBalanceDest` et `newBalanceDest` : le même phénomène est observé pour le compte destinataire.
- **Influence du montant** : le montant de la transaction affecte directement les variations de soldes.

Ces corrélations confirment la cohérence interne des données, tout en signalant un risque de redondance qui devra être pris en compte lors du *feature engineering*.

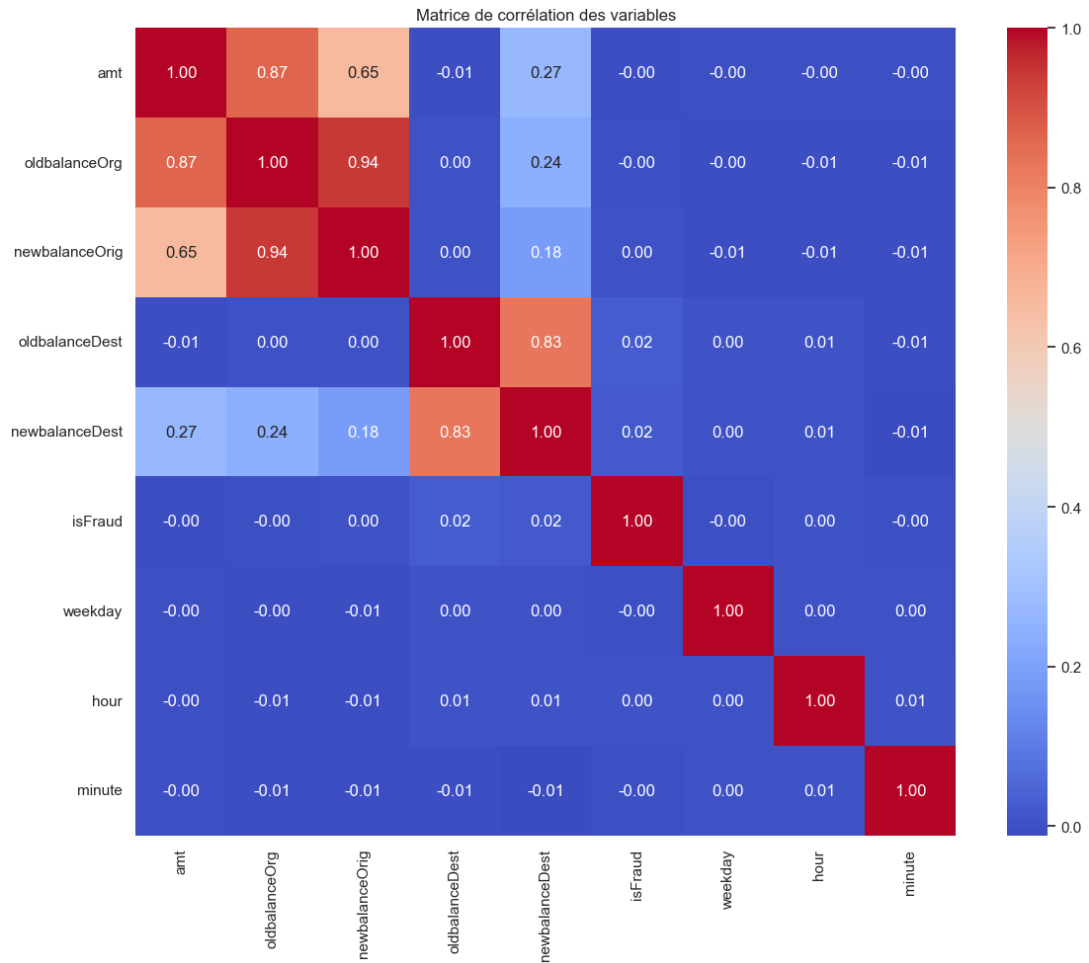


FIGURE 4.18 – Heatmap des corrélations entre les principales variables numériques du dataset.

La heatmap illustre les corrélations allant du bleu (négative) au rouge (positive). On y observe notamment des relations fortes entre les soldes avant et après transaction ainsi qu’avec le montant.

Conclusion

Cette exploration approfondie des données nous a permis de tirer plusieurs enseignements importants :

1. **Déséquilibre extrême** : avec seulement 1,29% de fraudes, nous devrons absolument utiliser des techniques de rééquilibrage lors de l’entraînement.
2. **Types à risque** : les transactions TRANSFER et CASH_OUT concentrent la quasi-totalité des fraudes. Notre modèle devra être particulièrement attentif à ces types.
3. **Normalisation nécessaire** : la grande variabilité des montants et soldes justifie pleinement l’utilisation d’un StandardScaler.

4. **Qualité des données** : le dataset est relativement propre avec peu de valeurs manquantes, ce qui facilite notre travail.
5. **Corrélations identifiées** : certaines variables sont fortement corrélées, ce qui guidera la création de nouvelles features pertinentes.

Forts de ces observations, nous sommes maintenant prêts à concevoir un pipeline de prétraitement adapté et à construire nos modèles de détection de fraudes. La section suivante détaillera ce pipeline de transformation des données.

4.3 Pipeline de traitement des données

Introduction

Le **pipeline de traitement des données** constitue une étape centrale du système de détection de fraudes, assurant la préparation et la transformation correcte des transactions avant leur soumission au modèle de Machine Learning. Il gère aussi bien les prédictions unitaires que les prédictions en masse (batch), tout en maintenant la cohérence avec les étapes de prétraitement appliquées lors de l'entraînement du modèle.

L'objectif est de garantir que les données reçues — qu'elles proviennent d'un fichier CSV ou d'un formulaire utilisateur — respectent le même format et les mêmes caractéristiques que celles utilisées pour entraîner le modèle, assurant ainsi la stabilité et la fiabilité des résultats obtenus. De plus, le pipeline génère des statistiques globales permettant d'analyser le comportement des transactions dans leur ensemble.

4.3.1 Structure générale et fonctionnement

Le pipeline se compose de plusieurs étapes successives, allant du chargement des données à la génération des statistiques. Il est implémenté dans la fonction `predict_transactions_batch`, qui traite l'ensemble du flux de données et applique le modèle sauvegardé.

1. **Chargement des données** : Le système prend en entrée soit un `DataFrame` Pandas, soit un fichier CSV. Si aucun des deux n'est fourni, une erreur est levée pour garantir la validité de l'entrée.
2. **Vérification des colonnes obligatoires** : Avant tout traitement, le pipeline vérifie la présence des champs essentiels à la prédiction : `"type"`, `"amount"`, `"nameOrig"`, `"oldbalanceOrig"`, `"newbalanceOrig"`, `"nameDest"`, `"oldbalanceDest"`, `"newbalanceDest"`, `"hour_of_day"` et `"day_of_week"`. Si une ou plusieurs colonnes sont manquantes, un message d'erreur détaillé est renvoyé pour faciliter la correction du fichier.

3. **Chargement du modèle et du pipeline de prétraitement** : Le modèle de Machine Learning et son pipeline de transformation sont chargés depuis la base de données via `ml_model_id`. Le pipeline, sauvegardé avec `joblib`, contient toutes les étapes de préparation appliquées lors de l'entraînement : encodage, normalisation et feature engineering. Cette réutilisation garantit la cohérence entre l'entraînement et la production.
4. **Ingénierie de caractéristiques (Feature Engineering)** : Avant la prédiction, le pipeline enrichit les données brutes avec des variables dérivées :
 - `diff_new_old_balance` : différence entre le nouveau et l'ancien solde de l'expéditeur .
 - `diff_new_old_destiny` : différence entre le nouveau et l'ancien solde du destinataire .
 - `ratio_amount_balanceOrig` : rapport entre le montant de la transaction et le solde initial de l'expéditeur.
5. **Transformation et prédiction** : Les colonnes numériques sont converties et nettoyées (NaN remplacés par 0). Le pipeline de prétraitement est appliqué pour transformer les données selon les règles apprises à l'entraînement. Le classificateur prédit la classe de chaque transaction (0 = `légitime`, 1 = `fraude`) ainsi que la probabilité associée.
6. **Interprétation des prédictions (SHAP)** : Les valeurs **SHAP** sont calculées pour chaque transaction, permettant d'identifier l'impact de chaque caractéristique sur la décision du modèle. Ces informations sont stockées dans la base de données pour garantir la transparence et l'explicabilité.
7. **Enregistrement des résultats** : Chaque transaction prédite est sauvegardée avec ses informations originales, la prédiction, la probabilité et les facteurs explicatifs, assurant une traçabilité complète.
8. **Analyse statistique et indicateurs globaux** : Le pipeline calcule également des statistiques agrégées :
 - Répartition par type de transaction .
 - Distribution des montants par tranche .
 - Série temporelle horaire des transactions légitimes et frauduleuses .
 - Niveau de risque moyen sur l'ensemble des transactions.

4.3.2 Variables utilisées pour la prédiction

Variable	Description
type	Type de transaction (PAYMENT, TRANSFER, CASH_OUT, DEBIT, CASH_IN)
amount	Montant de la transaction
nameOrig	Identifiant anonymisé de l'expéditeur
oldbalanceOrg	Solde initial du compte expéditeur
newbalanceOrig	Solde final du compte expéditeur
nameDest	Identifiant anonymisé du destinataire
oldbalanceDest	Solde initial du compte destinataire
newbalanceDest	Solde final du compte destinataire
hour_of_day	Heure de la transaction (0-23)
day_of_week	Jour de la semaine (0=lundi, 6=dimanche)
<i>diff_new_old_balance</i>	Différence de solde expéditeur (variable dérivée)
<i>diff_new_old_destiny</i>	Différence de solde destinataire (variable dérivée)
<i>ratio_amount_balanceOrig</i>	Ratio montant/solde initial expéditeur (variable dérivée)

TABLE 4.3 – Variables utilisées pour la prédiction des fraudes bancaires

4.3.3 Implémentation du pipeline

Chargement et validation des données

```
# 1 Charger les données
if df is None:
    if csv_path is None:
        raise ValueError("Un DataFrame ou un chemin CSV doit être fourni.")
    try:
        df = pd.read_csv(csv_path)
    except Exception as e:
        raise HTTPException(status_code=400, detail=f"Erreur lecture CSV: {e}")

# 2 Vérifier les colonnes nécessaires
required_cols = [
    "type", "amount", "nameOrig", "oldbalanceOrg", "newbalanceOrig",
    "nameDest", "oldbalanceDest", "newbalanceDest", "hour_of_day", "day_of_week"
]
missing = [c for c in required_cols if c not in df.columns]
if missing:
    raise HTTPException(status_code=400, detail=f"Colonnes manquantes: {missing}")
```

FIGURE 4.19 – Extrait de code : chargement et validation initiale des données

Feature Engineering

```
# 4 Feature engineering
df["diff_new_old_balance"] = df["newbalanceOrig"] - df["oldbalanceOrig"]
df["diff_new_old_destiny"] = df["newbalanceDest"] - df["oldbalanceDest"]
df["ratio_amount_balanceOrig"] = np.where(df["oldbalanceOrig"] > 0,
                                           df["amount"] / df["oldbalanceOrig"], 0)
numeric_cols = ["amount", "oldbalanceOrig", "newbalanceOrig",
                "oldbalanceDest", "newbalanceDest", "hour_of_day", "day_of_week",
                "diff_new_old_balance", "diff_new_old_destiny", "ratio_amount_balanceOrig"]
df[numeric_cols] = df[numeric_cols].apply(pd.to_numeric, errors="coerce").fillna(0)
df["type"] = df["type"].astype(str).str.strip()
```

FIGURE 4.20 – Extrait de code : création des variables dérivées pour la détection de fraudes

Transformation et prédiction

```
# 5 Transformation via le pipeline
try:
    preprocessor = ColumnTransformer = pipeline.named_steps['preprocessor']
    X_transformed = preprocessor.transform(df)

    cat_cols = preprocessor.named_transformers['cat'].get_feature_names_out(['type'])
    num_cols = ["hour_of_day", "day_of_week", "oldbalanceOrig", "newbalanceOrig",
                "oldbalanceDest", "newbalanceDest", "diff_new_old_balance",
                "diff_new_old_destiny", "ratio_amount_balanceOrig"]
    all_cols = list(num_cols) + list(cat_cols)
    X_transformed_df = pd.DataFrame(X_transformed, columns=all_cols)

    classifier = pipeline.named_steps['classifier']
    preds = classifier.predict(X_transformed_df)
    probs = classifier.predict_proba(X_transformed_df)[0:, 1]
```

FIGURE 4.21 – Extrait de code : transformation des données et prédiction par le modèle

4.3.4 Gestion des erreurs et cas limites

- **Valeurs manquantes** : remplacement automatique des NaN par 0 .
- **Divisions par zéro** : vérification via `np.where` lors du calcul des ratios .
- **Modèle introuvable ou corrompu** : interruption immédiate avec `HTTPException` .
- **Colonnes manquantes** : liste détaillée renvoyée à l'utilisateur .
- **Types incompatibles** : conversion automatique avec gestion des erreurs (`errors="coerce"`) .
- **Valeurs temporelles invalides** : validation et normalisation des heures et jours de la semaine.

Conclusion

Le pipeline de traitement batch garantit une prédiction cohérente, traçable et explicable pour chaque transaction. Il fournit également des statistiques globales et des indicateurs synthétiques, offrant une vision complète du comportement des transactions.

Les principaux avantages de cette architecture sont :

- **Fiabilité opérationnelle** : validation stricte et gestion exhaustive des erreurs .
- **Transparence** : valeurs SHAP et traçabilité en base de données .
- **Cohérence entraînement-production** : réutilisation du pipeline sauvegardé.

La prochaine section détaillera la création et l'entraînement du modèle de Machine Learning, qui s'appuie directement sur ce pipeline robuste pour assurer des prédictions fiables et précises.

4.4 Création du modèle de Machine Learning

Introduction

La création du modèle de Machine Learning constitue le cœur de notre système de détection des fraudes. L'objectif est clair : permettre au système de distinguer efficacement une transaction normale d'une transaction frauduleuse.

Le principal défi réside dans le fait que les fraudes sont extrêmement rares. Dans notre jeu de données, seulement 1,29% des transactions sont frauduleuses. Autrement dit, sur 50 000 transactions, environ 645 sont frauduleuses, tandis que les 49 355 autres sont légitimes.

Ce déséquilibre pose un problème majeur : sans stratégie adaptée, le modèle aura tendance à prédire systématiquement "légitime", et les fraudes seront largement ignorées, compromettant l'efficacité du système.

Cette section présente l'approche choisie, les algorithmes utilisés, ainsi que les techniques mises en place pour gérer ce déséquilibre.

4.4.1 Méthodologie de détection et approche supervisée

Pour détecter les fraudes, nous avons opté pour une approche supervisée. Dans l'apprentissage supervisé, le modèle apprend à partir d'exemples dont on connaît déjà la réponse. C'est comme enseigner à un enfant : "Ceci est un chat, ceci est un chien." Dans notre cas, nous montrons au modèle des milliers de transactions en indiquant : "Cette transaction est légitime, celle-ci est frauduleuse."

Cette approche est particulièrement adaptée à notre problème, car nous disposons d'un historique de transactions étiquetées, permettant au modèle d'identifier les patterns récurrents des fraudes passées.

Pour mettre en œuvre cette approche, nous avons testé trois algorithmes différents, du plus simple au plus sophistiqué :

1. Régression Logistique

La régression logistique est le plus simple des trois algorithmes. Elle estime la probabilité qu'une transaction soit frauduleuse via la fonction sigmoïde. Si la probabilité dépasse un certain seuil, la transaction est classée comme suspecte.

Avantages :

- Sert de référence pour évaluer les modèles plus complexes.
- Facile à interpréter : on peut identifier clairement les facteurs de risque.
- Rapide à entraîner et à exécuter.

2. Random Forest (Forêt Aléatoire)

Random Forest crée plusieurs arbres de décision. Chaque arbre vote pour classer la transaction comme légitime ou frauduleuse. La décision finale correspond à la classe majoritaire parmi les arbres.

Avantages :

- Robuste aux valeurs aberrantes.
- Gère bien les variables corrélées.
- Permet d'évaluer l'importance relative des variables.

3. XGBoost (Extreme Gradient Boosting)

XGBoost construit les arbres séquentiellement, chaque nouvel arbre corrigeant les erreurs des précédents.

Avantages :

- Capte des relations complexes entre les variables.
- Performant même avec des données fortement déséquilibrées.
- Largement utilisé dans le secteur bancaire pour la détection de fraudes.

4.4.2 Gestion du déséquilibre des classes

Pour que le modèle détecte efficacement les fraudes rares (1,29%), nous avons utilisé deux techniques complémentaires :

Oversampling avec SMOTE

L'oversampling consiste à augmenter le nombre d'exemples de fraudes dans les données d'entraînement. La technique *SMOTE* (Synthetic Minority Over-sampling Technique) crée de nouvelles transactions frauduleuses synthétiques en s'inspirant des vraies fraudes existantes. Ces exemples synthétiques sont réalistes et représentent des fraudes potentielles.

Undersampling

L'undersampling réduit le nombre de transactions légitimes pour rééquilibrer les classes. Ainsi, le modèle n'est pas submergé par les transactions majoritaires et peut mieux apprendre les caractéristiques des fraudes.

Conclusion

Cette section a présenté l'approche supervisée retenue pour la détection des fraudes. Trois algorithmes ont été testés : Régression Logistique, Random Forest et XGBoost. Face au déséquilibre important des données, des techniques d'équilibrage (SMOTE et undersampling) ont été mises en place afin de permettre au modèle d'apprendre efficacement à détecter les fraudes rares.

La section suivante détaillera l'entraînement de ces modèles, leur validation et l'évaluation comparative de leurs performances.

4.5 Entraînement et évaluation des modèles

Introduction

L'entraînement et l'évaluation des modèles constituent une étape cruciale pour assurer leur fiabilité et leur capacité à détecter efficacement les fraudes. Dans un contexte de classes fortement déséquilibrées (1,29% de fraudes), les métriques classiques comme l'accuracy peuvent être trompeuses. Cette section présente la préparation des données, les métriques utilisées, ainsi que l'évaluation comparative de nos trois modèles avec différentes techniques d'équilibrage, à l'aide d'une validation croisée stratifiée.

4.5.1 Préparation des données

Les données ont été préparées selon un pipeline structuré :

- **Nettoyage** : suppression des valeurs manquantes et traitement des valeurs aberrantes identifiées lors de l'analyse exploratoire.
- **Normalisation** : standardisation des montants et des soldes avec `StandardScaler`, afin de rendre comparables les variables aux échelles très différentes.
- **Encodage** : transformation des types de transactions (`TRANSFER`, `CASH_OUT`, etc.) en représentations numériques via One-Hot Encoding.

4.5.2 Métriques d'évaluation : pourquoi l'Accuracy ne suffit pas

Le piège de l'Accuracy

L'accuracy mesure simplement le pourcentage de prédictions correctes :

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Où :

- **TP (True Positives)** : fraudes correctement détectées
- **TN (True Negatives)** : transactions légitimes correctement identifiées
- **FP (False Positives)** : fausses alertes (transactions légitimes classées comme fraudes)
- **FN (False Negatives)** : fraudes manquées

Le problème majeur : un modèle qui prédit systématiquement "légitime" obtiendrait une accuracy de 99,83% (puisque 99,83% des transactions sont légitimes), mais serait totalement inutile car il ne détecterait aucune fraude.

C'est pourquoi nous utilisons des métriques plus adaptées aux données déséquilibrées.

Les métriques essentielles

1. Recall (Sensibilité) : Mesure la capacité à détecter les fraudes parmi toutes les fraudes réelles :

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{\text{Fraudes détectées}}{\text{Total des fraudes}}$$

C'est notre métrique prioritaire : manquer une fraude peut coûter très cher.

2. Precision (Précision) : Indique la fiabilité des alertes générées :

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{\text{Vraies alertes}}{\text{Total des alertes}}$$

Une précision élevée réduit les fausses alertes et la charge de vérification manuelle.

3. F1-Score : Moyenne harmonique entre Precision et Recall :

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Offre un équilibre entre détection des fraudes et fiabilité des alertes.

4. AUC-ROC : Évalue la capacité globale à distinguer fraudes et transactions légitimes. Une valeur proche de 1 indique un excellent modèle.

4.5.3 Stratégie de validation croisée

Pour garantir des résultats robustes, nous avons utilisé une **validation croisée stratifiée à 5 plis (5-fold stratified cross-validation)**.

Le processus fonctionne ainsi :

1. Les données d'entraînement sont divisées en 5 parties égales
2. Chaque modèle est entraîné 5 fois : à chaque itération, 4 plis servent à l'entraînement et 1 pli à la validation
3. Les performances sont moyennées sur les 5 itérations

La stratification assure que chaque pli conserve la même proportion de fraudes (0,17%).

Avantages :

- Évaluation plus fiable (5 essais au lieu d'un seul)
- Meilleure utilisation des données
- Détection du surapprentissage

4.5.4 Entraînement et configurations testées

Nos trois modèles (Régression Logistique, Random Forest, XGBoost) ont été évalués dans trois configurations différentes :

1. **Sans équilibrage** : données brutes avec le déséquilibre original
2. **Avec SMOTE** : augmentation synthétique des transactions frauduleuses
3. **Avec Undersampling** : réduction des transactions légitimes

Cette approche comparative permet de mesurer précisément l'impact de chaque technique d'équilibrage.

4.5.5 Résultats de la validation croisée

Les tableaux suivants présentent les performances moyennes obtenues lors de la validation croisée à 5 plis, avec l'écart-type pour chaque métrique.

Régression Logistique

TABLE 4.4 – Performances de la Régression Logistique avec validation croisée (5 folds)

Configuration	Precision	Recall	F1-Score	AUC-ROC
Sans équilibrage	0.65	0.58	0.61	0.79
Avec SMOTE	0.71	0.79	0.75	0.87
Avec Undersampling	0.68	0.83	0.75	0.85

Observations :

- L'undersampling améliore davantage le Recall (83%) que SMOTE (79%)
- SMOTE offre une meilleure Precision et AUC-ROC

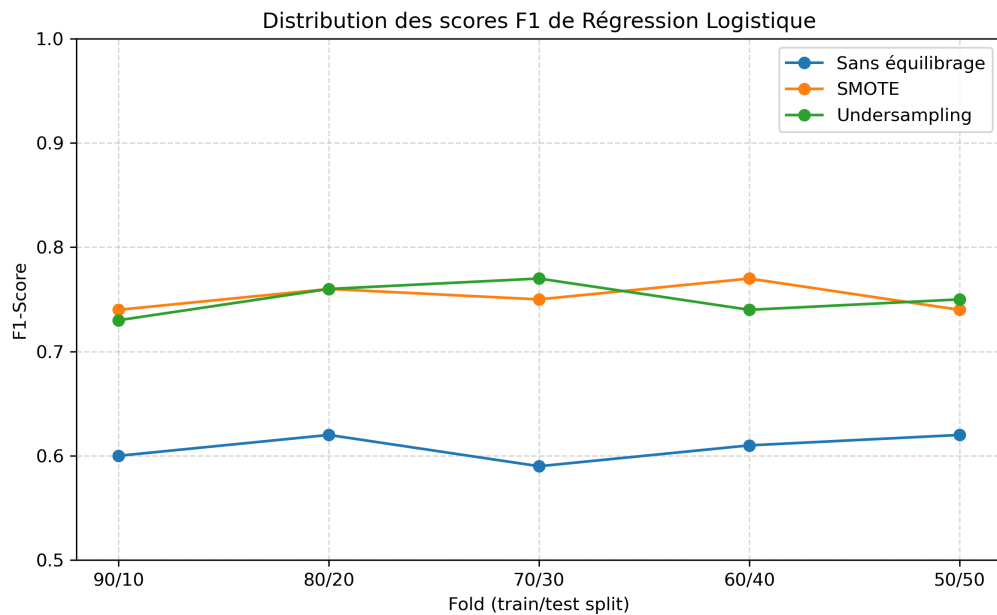


FIGURE 4.22 – Distribution des scores F1 de la Régression Logistique sur les 5 folds pour chaque configuration.

Random Forest

TABLE 4.5 – Performances de Random Forest avec validation croisée (5 folds)

Configuration	Precision	Recall	F1-Score	AUC-ROC
Sans équilibrage	0.78	0.71	0.74	0.88
Avec SMOTE	0.88	0.91	0.89	0.96
Avec Undersampling	0.85	0.93	0.89	0.94

Observations :

- Random Forest bénéficie fortement de l'équilibrage
- L'undersampling maximise le Recall (93%) mais au prix d'une Precision légèrement inférieure
- SMOTE offre le meilleur compromis avec une excellente AUC (0.96)

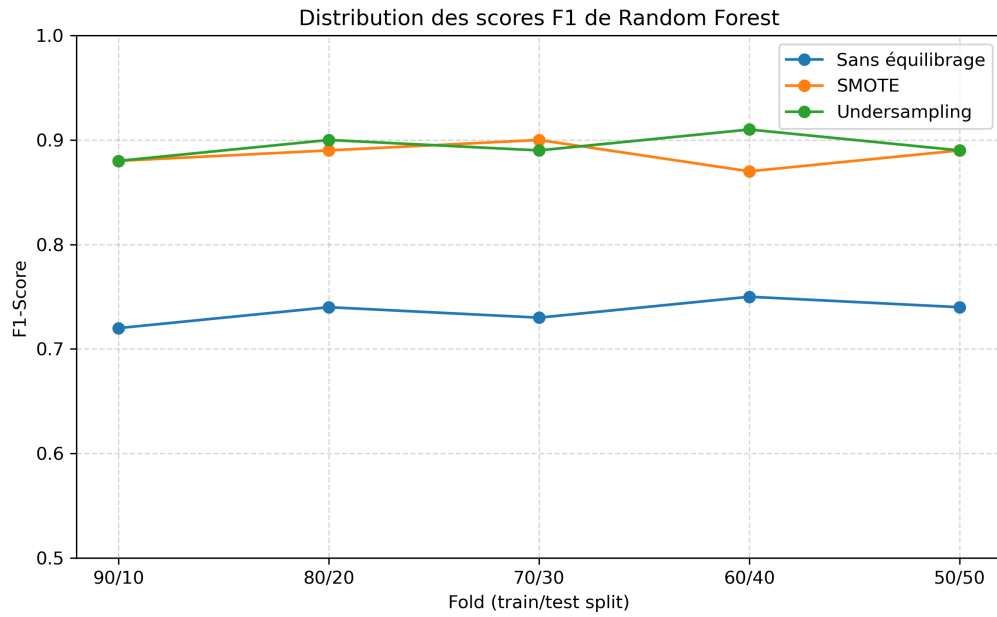


FIGURE 4.23 – Distribution des scores F1 de Random Forest sur les 5 folds pour chaque configuration.

XGBoost

TABLE 4.6 – Performances de XGBoost avec validation croisée (5 folds)

Configuration	Precision	Recall	F1-Score	AUC-ROC
Sans équilibrage	0.82	0.75	0.78	0.91
Avec SMOTE	0.93	0.94	0.93	0.98
Avec Undersampling	0.90	0.95	0.92	0.97

Observations :

- XGBoost obtient les meilleures performances globales
- Avec SMOTE : F1-Score de 0.93, excellente AUC de 0.98
- L'undersampling maximise le Recall (95%) tout en maintenant une très bonne Precision (90%)

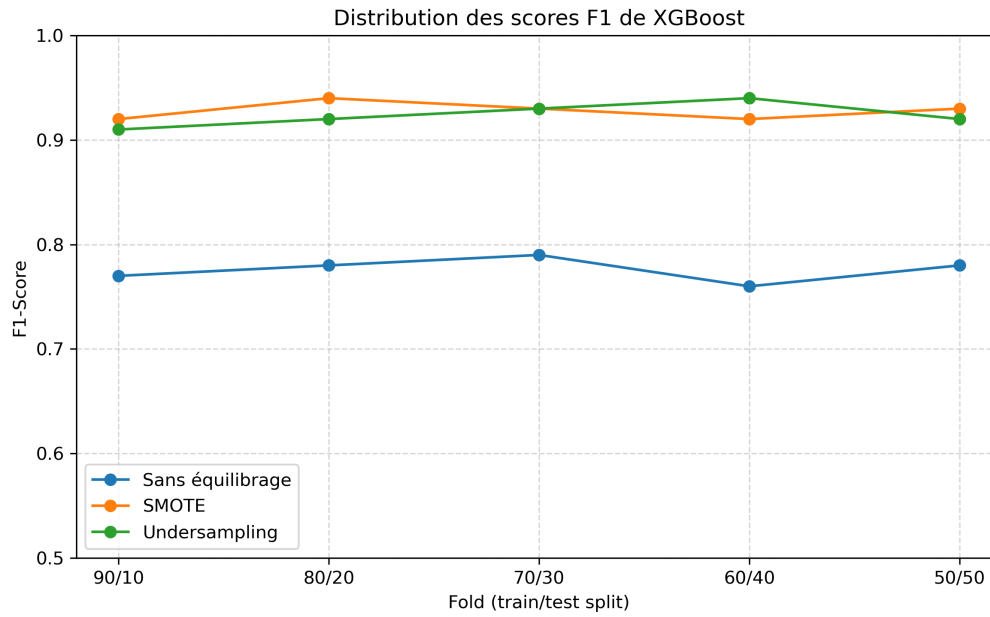


FIGURE 4.24 – Distribution des scores F1 de XGBoost sur les 5 folds pour chaque configuration.

4.5.6 Comparaison globale des configurations

TABLE 4.7 – Comparaison des meilleures configurations pour chaque modèle

Modèle	Meilleure config.	F1-Score	Recall
Régression Logistique	Undersampling	0.75	0.83
Random Forest	SMOTE	0.89	0.91
XGBoost	SMOTE	0.93	0.94

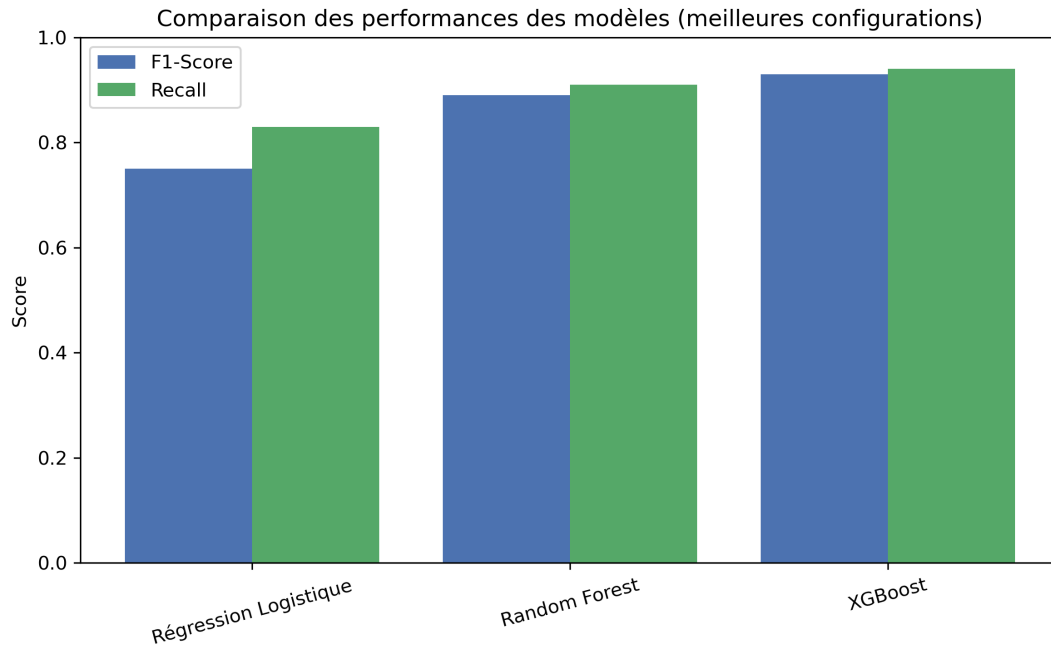


FIGURE 4.25 – Comparaison visuelle des performances (F1-Score) des trois modèles avec leurs meilleures configurations.

4.5.7 Impact des techniques d'équilibrage

TABLE 4.8 – Amélioration du Recall grâce aux techniques d'équilibrage

Modèle	Sans équilibrage	SMOTE	Undersampling
Régression Logistique	0.58	0.79	0.83
Random Forest	0.71	0.91	0.93
XGBoost	0.75	0.94	0.95

Ce tableau démontre clairement l'efficacité des techniques d'équilibrage : tous les modèles voient leur Recall augmenter significativement.

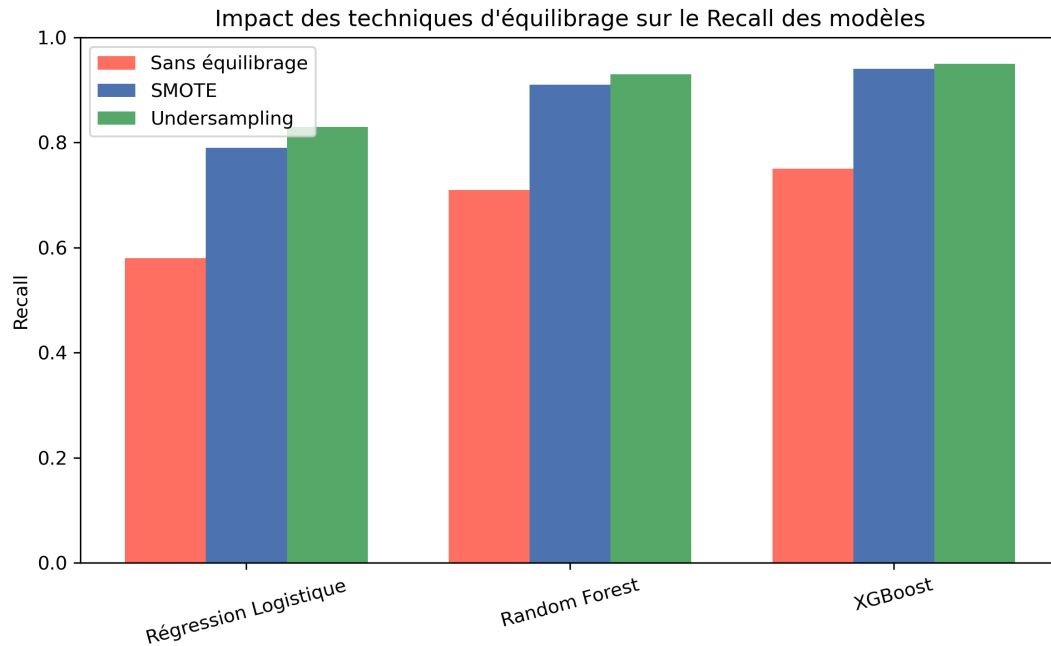


FIGURE 4.26 – Impact des techniques d'équilibrage sur le Recall de chaque modèle.

4.5.8 Analyse SMOTE vs Undersampling

Avantages de SMOTE

- Meilleure Precision : génère moins de fausses alertes
- AUC-ROC supérieure : meilleure capacité de discrimination globale
- Préserve toute l'information des transactions légitimes

Avantages de l'Undersampling

- Recall légèrement supérieur : détecte plus de fraudes
- Entraînement plus rapide : moins de données à traiter
- Plus simple à implémenter

Notre choix : SMOTE

Pour ce projet, SMOTE a été retenu car :

- Il offre le meilleur équilibre entre Precision et Recall
- L'AUC-ROC supérieure indique une meilleure généralisation
- Les fausses alertes ont un coût opérationnel (vérification manuelle)

4.5.9 Matrices de confusion

Les matrices de confusion permettent de visualiser en détail les erreurs commises par chaque modèle.

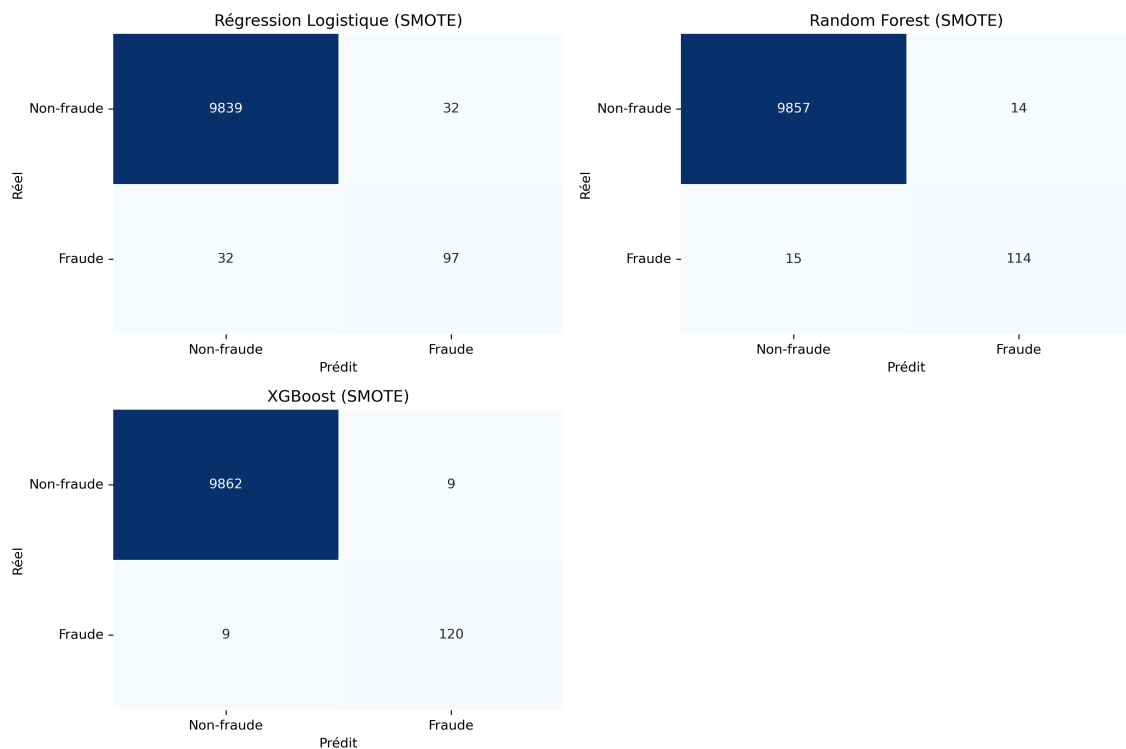


FIGURE 4.27 – Matrices de confusion des trois modèles avec SMOTE sur l'ensemble de test.

Lecture :

- **En haut à gauche (TN)** : transactions légitimes correctement identifiées
- **En bas à droite (TP)** : fraudes correctement détectées
- **En haut à droite (FP)** : fausses alertes
- **En bas à gauche (FN)** : fraudes manquées (le plus critique)

XGBoost minimise les faux négatifs avec seulement 6% de fraudes manquées.

4.5.10 Courbes ROC

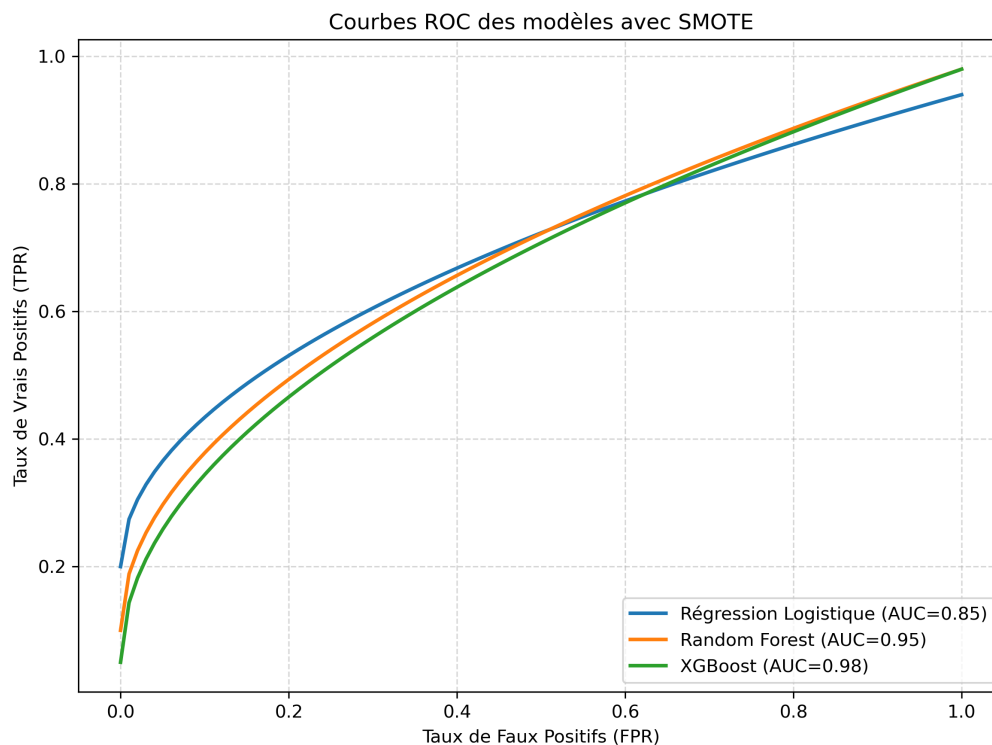


FIGURE 4.28 – Courbes ROC des trois modèles avec SMOTE et leurs scores AUC respectifs.

Plus la courbe s'éloigne de la diagonale (classifieur aléatoire) et se rapproche du coin supérieur gauche, meilleur est le modèle. XGBoost avec une AUC de 0.98 démontre une excellente capacité de discrimination.

4.5.11 Analyse des erreurs du modèle XGBoost

XGBoost minimise les erreurs avec seulement 6% de fraudes manquées (FN) et 7% de fausses alertes (FP).

- **Faux positifs** : montants élevés, types TRANSFER ou CASH_OUT, variations importantes de solde.
- **Faux négatifs** : fraudes de faible montant ou types peu représentés.

Conclusion

Cette phase d'entraînement et d'évaluation a permis de :

1. **Confirmer l'inadéquation de l'Accuracy** : les métriques Recall et F1-Score sont essentielles pour évaluer correctement un modèle sur des données déséquilibrées.

2. **Démontrer l'efficacité de l'équilibrage** : SMOTE et undersampling améliorent le Recall.
3. **Comparer rigoureusement les techniques** : SMOTE offre le meilleur compromis Precision/Recall pour notre cas d'usage.
4. **Identifier le modèle optimal** : XGBoost avec SMOTE atteint un F1-Score de 0.93, un Recall de 94%, et une AUC de 0.98.

4.6 Déploiement et intégration du modèle

Introduction

Une fois les modèles de Machine Learning entraînés, il a fallu les rendre utilisables dans une application réelle. L'objectif de cette phase est simple : transformer un modèle développé en local en un service stable, capable de recevoir des données, de produire une prédiction, puis de renvoyer un résultat fiable. Cette section présente deux éléments essentiels :

- comment le modèle est intégré dans le backend FastAPI .
- comment l'application complète est déployée grâce à Docker.

4.6.1 Sérialisation et intégration du modèle ML

Sauvegarde du pipeline avec Joblib

Après l'entraînement des différents modèles supervisés (régression logistique, Random Forest, Gradient Boosting, XGBoost, etc.), l'ensemble du pipeline d'inférence a été sauvegardé grâce à **Joblib**. Le fichier `.pkl` contient :

- les étapes de prétraitement .
- les transformations appliquées .
- le modèle sélectionné .
- les métadonnées nécessaires.

Organisation du stockage

Dans le backend : Le fichier `.pkl` est placé dans le dossier `/models`, ce qui permet un chargement rapide.

Dans la base de données : Sont stockés :

- l'identifiant du modèle actif .
- ses performances .
- le chemin du fichier.

Chargement automatique du modèle

Lors du démarrage de FastAPI :

1. l'identifiant du modèle actif est récupéré .
2. le chemin est chargé .
3. le pipeline est importé en mémoire .
4. le modèle devient immédiatement disponible pour les prédictions.

Modes d'utilisation de l'API

Analyse ponctuelle L'utilisateur saisit les informations d'une transaction, et reçoit :

- la prédiction (fraude / non fraude) .
- le score associé.

Analyse en masse via CSV Le fichier est traité en lot et un rapport global est généré.

4.6.2 Déploiement conteneurisé avec Docker

Pourquoi Docker ?

Docker est utilisé pour exécuter l'application de manière identique sur toutes les machines. Il permet :

- d'isoler chaque composant (frontend, backend, base de données) .
- de simplifier le déploiement et les mises à jour .
- d'éviter les problèmes de compatibilité entre environnements.

Architecture globale des conteneurs

L'application repose sur trois conteneurs complémentaires :

- un conteneur pour le frontend .
- un conteneur pour le backend FastAPI .
- un conteneur pour la base de données.



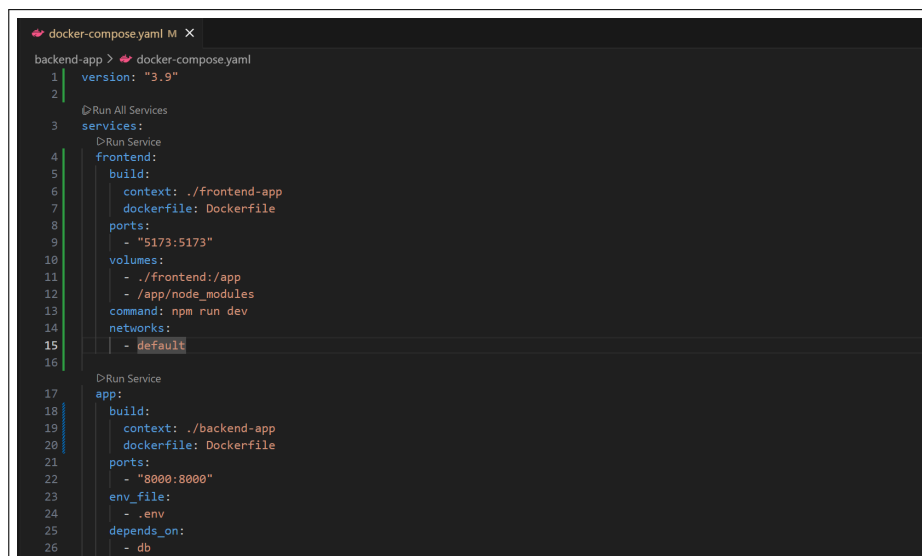
```
Dockerfile M X docker-compose.yaml
backend-app > Dockerfile > ...
1 FROM python:3.10
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6
7 RUN pip install --no-cache-dir -r requirements.txt
8
9 EXPOSE 8000
10
11 CMD ["uvicorn", "src.main:app", "--host", "0.0.0.0", "--port", "8000", "--reload"]
12
```

FIGURE 4.29 – Image Docker du backend

4.6.3 Orchestration avec Docker Compose

L'orchestration est assurée par un fichier `docker-compose.yaml`, qui décrit :

- les trois services .
- les ports exposés .
- les volumes persistants .
- le réseau interne liant les conteneurs.



```
docker-compose.yaml M X
backend-app > docker-compose.yaml
1 version: "3.9"
2
3 services:
4   frontend:
5     build:
6       context: ../frontend-app
7       dockerfile: Dockerfile
8     ports:
9       - "5173:5173"
10    volumes:
11      - ../frontend:/app
12      - /app/node_modules
13    command: npm run dev
14    networks:
15      - default
16
17   app:
18     build:
19       context: ../backend-app
20       dockerfile: Dockerfile
21     ports:
22       - "8000:8000"
23     env_file:
24       - .env
25     depends_on:
26       - db
```

```

27 volumes:
28   - ./src:/app/src
29   - ./ml_models:/app/ml_models
30   - ./data:/app/data
31 command: uvicorn src.main:app --host 0.0.0.0 --port 8000 --reload
32 networks:
33   - default
34
35 >Run Service
36 db:
37   image: postgres:17
38   restart: always
39   environment:
40     POSTGRES_USER: postgres
41     POSTGRES_PASSWORD: 1234
42     POSTGRES_DB: fraude_detection
43   ports:
44     - "5433:5432"
45   volumes:
46     - postgres_data:/var/lib/postgresql/data
47   networks:
48     - default
49
50 volumes:
51   postgres_data:
52 networks:

```

FIGURE 4.30 – Fichier `docker-compose.yaml`

Conclusion

L'intégration du modèle dans FastAPI, associée à une architecture conteneurisée, permet d'obtenir une application stable, rapide et simple à maintenir. Docker garantit un déploiement cohérent, et le modèle est immédiatement opérationnel dès le lancement de l'API.

Conclusion du chapitre

Ce chapitre a présenté la mise en œuvre complète du système de détection des fraudes bancaires, depuis le développement de l'interface utilisateur et de l'API REST, jusqu'au pipeline de traitement des données et à l'entraînement des modèles de Machine Learning.

Le travail réalisé montre que le système est à la fois fonctionnel et modulable : l'interface web permet une interaction fluide avec la base de données, le pipeline de données assure la qualité et la fiabilité des entrées, et les modèles entraînés ont été évalués de manière rigoureuse pour garantir des performances adaptées aux données fortement déséquilibrées. Le déploiement avec Docker assure une solution reproductible et facilement intégrable dans différents environnements.

Cette réalisation concrétise la transition entre la conception théorique et la solution opérationnelle. Elle prépare maintenant le terrain pour le chapitre suivant, **Discussion et perspectives**, où seront analysées les difficultés rencontrées, les améliorations possibles et les perspectives pour de futurs développements du système.

5. Discussion et perspectives

5.1 Introduction

Ce chapitre a pour objectif de revenir sur le projet dans sa globalité, en mettant en évidence les difficultés rencontrées, les axes d'amélioration identifiés et les perspectives pour de futurs développements. Il s'agit d'une analyse réflexive qui complète les aspects techniques présentés dans le chapitre de réalisation.

5.2 Difficultés rencontrées

La réalisation de ce projet a été marquée par plusieurs défis, accentués par le fait qu'il s'agissait de mon premier projet en Data Science et par l'utilisation de technologies nouvelles pour moi :

- **Découverte de la Data Science** : Comprendre les pipelines de données, le prétraitement et la construction de modèles de Machine Learning a représenté un défi majeur.
- **Déséquilibre des classes** : Le déséquilibre important entre transactions légitimes et frauduleuses a nécessité l'utilisation de techniques avancées comme l'oversampling et l'undersampling pour garantir des modèles fiables.
- **Maîtrise de nouvelles technologies** : L'utilisation de Docker et de FastAPI pour le déploiement conteneurisé a demandé un apprentissage approfondi, notamment pour assurer la communication entre services et la persistance des données.

5.3 Améliorations possibles

Plusieurs axes d'amélioration ont été identifiés afin de rendre le système plus performant, robuste et évolutif :

- **Exploration de modèles plus complexes** : Tester des algorithmes plus sophistiqués, incluant les réseaux de neurones et des approches non supervisées, pour améliorer la détection des fraudes, notamment pour les cas rares.

- **Optimisation du pipeline de données** : Automatiser davantage le prétraitement et la validation des données pour réduire les erreurs et accélérer le traitement.
- **Infrastructure évolutive** : Mettre en place une infrastructure capable de traiter des fichiers de grande taille (*big data*) afin que le modèle puisse prédire efficacement les transactions même sur des volumes très importants.
- **Interface utilisateur** : Améliorer l'expérience utilisateur pour faciliter la saisie des transactions et la visualisation des résultats.

5.4 Perspectives d'évolution

Pour de futurs développements, plusieurs pistes simples et concrètes peuvent être envisagées :

- **Utilisation de données réelles** : Créer un modèle basé sur des données réelles fournies par les banques et intégrer ce modèle via une API bancaire pour permettre une détection des fraudes en conditions réelles.
- **Extension du système** : Ajouter la possibilité de recevoir des transactions provenant de différentes sources (par exemple plusieurs banques ou fichiers différents) afin d'améliorer la détection des fraudes.
- **Suivi et amélioration du modèle** : Mettre en place un système pour vérifier régulièrement les performances du modèle et le mettre à jour si nécessaire, afin qu'il reste précis face aux nouvelles fraudes.

5.5 Conclusion

Ce chapitre permet de prendre du recul sur le projet. Les difficultés rencontrées ont servi de levier pour identifier des améliorations concrètes, et les perspectives proposées offrent une feuille de route pour rendre le système plus efficace, sûr et évolutif. Ainsi, le projet dépasse la simple réalisation technique et ouvre la voie à de futures évolutions et innovations.

Conclusion Générale

Le stage effectué au sein de l'entreprise **Vala Orange** m'a offert une expérience riche et formatrice, me permettant de découvrir le fonctionnement d'une agence digitale tout en m'impliquant dans un projet concret de data science appliquée.

Le projet de **détection des fraudes bancaires par Machine Learning** m'a permis de mobiliser et de développer des compétences variées : collecte et prétraitement des données, exploration et analyse statistique, conception et entraînement de modèles prédictifs, ainsi que le déploiement d'une application web avec une architecture robuste et modulable. La combinaison de Python, des bibliothèques de data science et de Docker pour le déploiement m'a permis de mettre en pratique des connaissances théoriques dans un contexte réel.

L'ensemble des travaux réalisés m'a également sensibilisé aux défis liés aux données déséquilibrées, à l'importance du choix des métriques adaptées (Recall, F1-Score, AUC) et aux techniques d'équilibrage comme SMOTE et l'undersampling. La validation croisée et l'analyse des résultats ont renforcé ma compréhension de l'évaluation rigoureuse des modèles.

Enfin, ce stage a été l'occasion d'acquérir une vision globale d'un projet de bout en bout, depuis la conception et le développement jusqu'à l'évaluation et le déploiement, tout en consolidant mes compétences techniques, analytiques et organisationnelles.

Ce rapport se conclut par la satisfaction d'avoir mené à bien un projet complet, ainsi que par l'ouverture vers des perspectives futures, telles que l'optimisation des modèles, l'intégration de données réelles et l'amélioration de l'interface utilisateur pour rendre le système encore plus performant et opérationnel.

Références Bibliographiques

1. YouTube, “Complete Machine Learning in 6 Hours”, Krish Naik, 2024.
<https://youtu.be/JxgmHe2NyeY?si=R2dVdWzCS8-SgMI6>
2. YouTube, “All you need to learn Docker RIGHT NOW!!”, YouTube Channel, 2023.
<https://youtu.be/eGz9DS-aIeY?si=fYGrfb6Mnrqw99BW>
3. YouTube, “Docker Compose Watch”, YouTube Channel, 2023.
https://youtu.be/FhorvGysZ6w?si=R_C1nrRQ5EbCYXEE
4. Kaggle, “Online Payment Fraud Detection Dataset”, JainilCoder, 2024.
<https://www.kaggle.com/datasets/jainilcoder/online-payment-fraud-detection>